SQL INTERVIEW PREPARATION GUIDE

150 Questions & Comprehensive Answers

MySQL | On-Campus & Company Assessment Ready

Barclays | JP Morgan | UBS | HackerRank | LeetCode

SECTION 0 — 50 Foundational Classic Patterns (2nd Highest, Nth Highest, Self-Join, Duplicates…)

SECTIONS 1–5 — 100 Core Interview Questions (Easy to Hard)

SECTIONS 6–10 — Advanced Company Assessment Level (Barclays / JP Morgan / UBS)

Each question: Problem + Schema → Multiple Solutions → Line-by-Line Explanation → Intuition → Pattern Tip

How to Use This Guide

This guide contains 150 MySQL interview questions across three tiers:

SECTION 0 — 50 Foundational questions: every classic pattern asked in campus and entry-level interviews. 2nd/Nth highest salary, employees > manager, duplicates, self-join, window functions, aggregations, string/date functions.

SECTIONS 1–5 — 100 questions from Easy to Hard: JOINs, GROUP BY, HAVING, subqueries, window functions, date operations, conditional logic, and all common LeetCode/HackerRank SQL patterns.

SECTIONS 6–10 — Advanced company-assessment style: financial analytics, risk, compliance, trading, and operational SQL modelled on actual Barclays, JP Morgan, and UBS assessments.

Each question includes: (1) Problem statement + table schema, (2) Multiple solution approaches, (3) Line-by-line explanation, (4) INTUITION box, (5) PATTERN TIP.

SECTION 0

Foundational SQL Patterns

50 Classic Questions Every Developer Must Know

SECTION 0.1

Classic Patterns I

2nd/Nth Highest, Self-Join, Duplicates, NULL Handling

QF1. Second Highest Salary

Foundational
JP Morgan, Barclays, UBS, Infosys, TCS

PROBLEM STATEMENT

Write a query to find the second highest salary from the Employee table. Return NULL if it does not exist.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
salary	INT	Employee salary

SOLUTION 1: Subquery + MAX (Most Common Interview Answer)

```
SELECT MAX(salary) AS SecondHighestSalary
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Explanation:

- Inner subquery: finds the maximum salary.
- Outer WHERE salary < max: excludes the highest salary.
- Outer MAX: finds the highest of the remaining = 2nd highest.
- Returns NULL automatically if only one distinct salary exists.

SOLUTION 2: LIMIT OFFSET

```
SELECT DISTINCT salary AS SecondHighestSalary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Explanation:

- DISTINCT: removes duplicate salary values.
- ORDER BY DESC: highest salary first.
- LIMIT 1 OFFSET 1: skip 1 row (highest), take the next = 2nd highest.

SOLUTION 3: DENSE_RANK Window Function

```
SELECT salary AS SecondHighestSalary
```

```
FROM (
  SELECT salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM Employee
) t
WHERE rnk = 2
LIMIT 1;
```

Explanation:
• DENSE_RANK: assigns 1 to highest, 2 to 2nd highest (no gaps for ties).
• WHERE rnk = 2: second highest salary.
• LIMIT 1: if multiple employees share 2nd rank, return value once.

INTUITION: 2nd highest = exclude MAX then take MAX again. Or DENSE_RANK = 2. DISTINCT is critical when multiple employees share salaries. This appears in virtually every SQL interview.

PATTERN TIP: DENSE_RANK vs RANK: 100,100,90 -> DENSE_RANK: 1,1,2. RANK: 1,1,3. Use DENSE_RANK for 'Nth highest' problems.

QF2. Nth Highest Salary

Foundational
JP Morgan, Barclays, Wipro, Cognizant

PROBLEM STATEMENT

Write a query to find the Nth highest distinct salary. Return NULL if there is no Nth highest.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
salary	INT	Employee salary

SOLUTION 1: LIMIT OFFSET (N-1)

```
-- For N=3 (3rd highest):
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
-- General formula: OFFSET = N - 1
```

Explanation:
• DISTINCT: treat equal salaries as one rank.
• ORDER BY DESC: highest salary first.
• OFFSET N-1: skip N-1 rows. N=1 -> skip 0. N=3 -> skip 2.
• LIMIT 1: take exactly the Nth row.

SOLUTION 2: DENSE_RANK = N

```
SELECT salary
FROM (
  SELECT salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM Employee
```

```
) t
WHERE rnk = 3  -- Replace 3 with N
LIMIT 1;
```

Explanation:
• DENSE_RANK: continuous ranks 1,2,3 even with ties.
• WHERE rnk = N: the Nth distinct salary directly.

INTUITION: Nth highest generalises 2nd highest. OFFSET N-1 skips exactly N-1 rows. DENSE_RANK = N is direct. Both need DISTINCT to handle salary ties correctly.

PATTERN TIP: LIMIT 1 OFFSET N-1: 1st highest = OFFSET 0, 2nd = OFFSET 1, 3rd = OFFSET 2. Pattern: offset is always N minus 1.

QF3. Employees Earning More Than Their Manager

Foundational
JP Morgan, Barclays, LeetCode Classic

PROBLEM STATEMENT
Find the names of all employees who earn more than their direct manager.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Employee name
salary	INT	Salary
managerId	INT	Manager ID (references id in same table)

SOLUTION 1: Self-Join (Standard)
```
SELECT e.name AS Employee
FROM Employee e
JOIN Employee m ON e.managerId = m.id
WHERE e.salary > m.salary;
```

Explanation:
• Self-join: same table aliased as e (employee) and m (manager).
• e.managerId = m.id: links each employee to their manager's row.
• WHERE e.salary > m.salary: employee earns more than their manager.

SOLUTION 2: Correlated Subquery
```
SELECT name AS Employee
FROM Employee e1
WHERE salary > (
  SELECT salary FROM Employee e2
  WHERE e2.id = e1.managerId
);
```

Explanation:
• For each employee row, subquery fetches their manager's salary.

• Compare employee salary to manager salary.

INTUITION: When a table references itself (managerId -> id), use a SELF-JOIN. Alias the table twice: once as employee (e), once as manager (m). Then compare their salaries.

PATTERN TIP: Self-join pattern: JOIN same_table alias2 ON alias1.foreign_key = alias2.primary_key. Meaningful aliases (e for employee, m for manager) make the query readable.

QF4. Find All Duplicate Records

Foundational
Amazon, TCS, Infosys

PROBLEM STATEMENT

Find all email addresses that appear more than once in the Person table.

Table: Person

Column Name	Type	Description
id	INT	Unique person ID
email	VARCHAR	Email address

SOLUTION 1: GROUP BY + HAVING COUNT > 1

```
SELECT email
FROM Person
GROUP BY email
HAVING COUNT(*) > 1;
```

Explanation:

• GROUP BY email: groups all rows with the same email together.
• HAVING COUNT(*) > 1: only groups with more than one row = duplicates.
• HAVING is used (not WHERE) because we filter on an aggregated value.

INTUITION: Duplicates = GROUP BY the column, HAVING COUNT > 1. HAVING filters groups after aggregation. This is one of the most asked SQL patterns.

PATTERN TIP: WHERE filters rows before aggregation. HAVING filters groups after aggregation. Rule: if you need to filter on COUNT/SUM/AVG, use HAVING.

QF5. Customers Who Never Ordered

Foundational
LeetCode, Infosys, Wipro

PROBLEM STATEMENT

Find the names of all customers who have never placed an order.

Table: Customers

Column Name	Type	Description
id	INT	Customer ID
name	VARCHAR	Customer name

Table: Orders

Column Name	Type	Description
id	INT	Order ID
customerId	INT	FK to Customers

SOLUTION 1: LEFT JOIN + IS NULL

```
SELECT c.name AS Customers
FROM Customers c
LEFT JOIN Orders o ON c.id = o.customerId
WHERE o.id IS NULL;
```

Explanation:

- LEFT JOIN: returns ALL customers, with NULLs where no order exists.
- WHERE o.id IS NULL: keeps only customers with no matching order.

SOLUTION 2: NOT IN

```
SELECT name AS Customers
FROM Customers
WHERE id NOT IN (
  SELECT customerId FROM Orders
);
```

Explanation:

- Subquery: all customer IDs who have at least one order.
- NOT IN: customers not in that list = never ordered.
- Warning: NOT IN gives wrong results if subquery contains NULL values.

INTUITION: Find records with no match in another table: LEFT JOIN IS NULL or NOT IN. LEFT JOIN IS NULL is safer when the foreign key column may contain NULLs.

PATTERN TIP: Prefer LEFT JOIN IS NULL over NOT IN. NOT IN with any NULL in the subquery returns zero rows (NULL comparison = UNKNOWN). NOT EXISTS also works and handles NULLs correctly.

QF6. Highest Salary in Each Department

Foundational
Amazon, JP Morgan, HackerRank

PROBLEM STATEMENT

Find the highest salary in each department.

Table: Employee

Column Name	Type	Description

id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary
department	VARCHAR	Department name

SOLUTION 1: GROUP BY + MAX

```
SELECT department,
       MAX(salary) AS highest_salary
FROM Employee
GROUP BY department
ORDER BY highest_salary DESC;
```

Explanation:

- GROUP BY department: one group per department.
- MAX(salary): the highest salary within each group.

INTUITION: Per-group maximum: GROUP BY the grouping column, apply MAX(). This is the simplest aggregation pattern.

PATTERN TIP: MAX(), MIN(), AVG(), SUM(), COUNT() all work with GROUP BY. Each returns one value per group.

QF7. Count Employees per Department

Foundational
General, TCS, Wipro

PROBLEM STATEMENT

Count the number of employees in each department, ordered by count descending.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
department	VARCHAR	Department name

SOLUTION 1: GROUP BY + COUNT

```
SELECT department,
       COUNT(*) AS employee_count
FROM Employee
GROUP BY department
ORDER BY employee_count DESC;
```

Explanation:

- GROUP BY department: one group per department.
- COUNT(*): number of rows in each group = number of employees.
- ORDER BY employee_count DESC: most populated department first.

INTUITION: Count per group = GROUP BY + COUNT(*). This is the most fundamental SQL aggregation.

PATTERN TIP: COUNT(*) counts all rows including NULLs. COUNT(column) counts only non-NULL values in that column. For employee count, use COUNT(*).

QF8. Find the Maximum Salary

Foundational
General

PROBLEM STATEMENT

Find the employee(s) with the highest salary in the company.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary

SOLUTION 1: Subquery

```
SELECT name, salary
FROM Employee
WHERE salary = (SELECT MAX(salary) FROM Employee);
```

Explanation:

- Subquery: returns the maximum salary (scalar value).
- Outer WHERE: returns all employees matching that salary.
- Handles ties: multiple employees at same max salary are all returned.

SOLUTION 2: ORDER BY LIMIT

```
SELECT name, salary
FROM Employee
ORDER BY salary DESC
LIMIT 1;
```

Explanation:

- ORDER BY DESC: highest salary first.
- LIMIT 1: only the first row.
- Note: if there are ties, only one row is returned (unlike subquery approach).

INTUITION: Max salary employee: subquery for MAX value, then filter WHERE salary = MAX. This handles ties correctly. ORDER BY LIMIT 1 returns only one row even with ties.

PATTERN TIP: To handle ties (multiple employees with same highest salary): use WHERE salary = (SELECT MAX(salary)). To return just one: use ORDER BY DESC LIMIT 1.

QF9. Select Distinct Values

Foundational
General

PROBLEM STATEMENT

Find all distinct department names from the Employee table.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
department	VARCHAR	Department name

SOLUTION 1: SELECT DISTINCT

```
SELECT DISTINCT department
FROM Employee
ORDER BY department;
```

Explanation:

- DISTINCT: removes duplicate department values.
- Returns each unique department name exactly once.

INTUITION: DISTINCT removes duplicate rows from the result. It applies to the entire row (all selected columns combined).

PATTERN TIP: SELECT DISTINCT col1, col2: removes rows where BOTH col1 AND col2 are the same. Not just col1. For single-column uniqueness, DISTINCT works perfectly.

QF10. Filter with Multiple Conditions (AND/OR)

Foundational
General

PROBLEM STATEMENT

Find all employees in the IT department earning more than 50000.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary
department	VARCHAR	Department

SOLUTION 1: WHERE with AND

```
SELECT id, name, salary, department
FROM Employee
WHERE department = 'IT'
  AND salary > 50000
```

```
ORDER BY salary DESC;
```

Explanation:
• AND: both conditions must be true.
• department = 'IT': string comparison (case-sensitive in MySQL by default).
• salary > 50000: numeric comparison.

INTUITION: Multiple conditions: AND (both must be true), OR (either must be true). Use parentheses for complex combinations: (A OR B) AND C.

PATTERN TIP: AND has higher precedence than OR. (A OR B AND C) = (A OR (B AND C)). Use explicit parentheses to avoid ambiguity: (A OR B) AND C.

SECTION 0.2

Classic Patterns II

BETWEEN, LIKE, IN, ORDER BY, LIMIT, All Five Aggregates

QF11. Find Employees with NULL Manager

Foundational
General, TCS

PROBLEM STATEMENT

Find all employees who do not have a manager (managerId is NULL) -- these are the top-level managers.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
managerId	INT	Manager ID (NULL for top-level)

SOLUTION 1: IS NULL

```
SELECT id, name
FROM Employee
WHERE managerId IS NULL
ORDER BY name;
```

Explanation:
• IS NULL: checks for NULL values.
• Cannot use = NULL (that always returns false in SQL).
• WHERE managerId IS NULL: finds rows where managerId has no value.

INTUITION: NULL comparisons: always use IS NULL or IS NOT NULL. Never use = NULL or != NULL -- these always return FALSE/UNKNOWN in SQL.

PATTERN TIP: NULL rules: NULL = NULL is FALSE. NULL != NULL is FALSE. Only IS NULL returns TRUE for NULL values. This trips up many beginners.

QF12. BETWEEN Operator (Range Filter)

Foundational
General

PROBLEM STATEMENT

Find all employees with salary between 40000 and 80000 (inclusive).

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary

SOLUTION 1: BETWEEN...AND

```
SELECT id, name, salary
FROM Employee
WHERE salary BETWEEN 40000 AND 80000
ORDER BY salary;
```

Explanation:

- BETWEEN low AND high: inclusive on both ends.
- Equivalent to: salary >= 40000 AND salary <= 80000.

INTUITION: BETWEEN is inclusive: BETWEEN 40000 AND 80000 includes both 40000 and 80000. Works on numbers, dates, and strings.

PATTERN TIP: BETWEEN is always inclusive. For dates: BETWEEN '2023-01-01' AND '2023-12-31' includes both boundary dates. NOT BETWEEN excludes the range.

QF13. LIKE Operator (Pattern Matching)

Foundational
General, Wipro

PROBLEM STATEMENT

Find all employees whose name starts with the letter A.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary

SOLUTION 1: LIKE with % wildcard

```
SELECT id, name, salary
FROM Employee
WHERE name LIKE 'A%'
ORDER BY name;
```

Explanation:

• LIKE 'A%': name starts with A (% matches any sequence of characters).
• % can be placed anywhere: 'A%' = starts with A, '%A' = ends with A, '%A%' = contains A.

INTUITION: LIKE for pattern matching: % = any sequence of characters, _ = any single character. Case-insensitive by default in MySQL for non-binary columns.

PATTERN TIP: LIKE wildcards: % = zero or more characters. _ = exactly one character. 'J_hn' matches 'John', 'Jahn'. 'A%n' matches 'Aaron', 'Akon'.

QF14. IN Operator (List Filter)

Foundational
General

PROBLEM STATEMENT

Find all employees who work in IT, HR, or Finance departments.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
department	VARCHAR	Department

SOLUTION 1: IN with value list

```
SELECT id, name, department
FROM Employee
WHERE department IN ('IT', 'HR', 'Finance')
ORDER BY department, name;
```

Explanation:

• IN ('IT','HR','Finance'): matches any of the listed values.
• Equivalent to: department='IT' OR department='HR' OR department='Finance'.
• More readable than chained OR conditions.

INTUITION: IN is shorthand for multiple OR conditions. IN ('a','b','c') = col='a' OR col='b' OR col='c'. Also works with subqueries: IN (SELECT ...).

PATTERN TIP: IN with subquery: WHERE id IN (SELECT manager_id FROM ...). NOT IN: excludes the list. Warning: NOT IN with NULLs in the list returns no rows.

QF15. ORDER BY Multiple Columns

Foundational
General

PROBLEM STATEMENT

List all employees ordered by department name ascending, then by salary descending within each department.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary
department	VARCHAR	Department

SOLUTION 1: ORDER BY col1 ASC, col2 DESC

```
SELECT id, name, department, salary
FROM Employee
ORDER BY department ASC, salary DESC;
```

Explanation:

- ORDER BY department ASC: primary sort by department alphabetically.
- salary DESC: secondary sort - within same department, highest salary first.
- ASC is the default; only DESC needs to be specified explicitly.

INTUITION: Multi-column ORDER BY: first column is primary sort, second is tiebreaker, etc. Each can have its own ASC/DESC direction.

PATTERN TIP: ASC is default - you don't need to write it. Only write DESC when needed. ORDER BY col1, col2 DESC: col1 is ASC, col2 is DESC.

QF16. LIMIT and OFFSET (Pagination)

Foundational
General

PROBLEM STATEMENT

Find the top 5 highest-paid employees. Then find employees ranked 6th to 10th.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary

SOLUTION 1: LIMIT and LIMIT with OFFSET

```
-- Top 5 highest paid
SELECT id, name, salary
FROM Employee
```

```
ORDER BY salary DESC
LIMIT 5;

-- 6th to 10th highest paid
SELECT id, name, salary
FROM Employee
ORDER BY salary DESC
LIMIT 5 OFFSET 5;
```

Explanation:
• LIMIT 5: return only 5 rows.
• LIMIT 5 OFFSET 5: skip 5 rows, then return 5 rows.
• Page N (1-indexed): LIMIT page_size OFFSET (N-1)*page_size.

INTUITION: LIMIT N: return N rows. LIMIT N OFFSET M: skip M rows, return N rows. This is how pagination works in SQL.

PATTERN TIP: Page formula: Page 1 = OFFSET 0, Page 2 = OFFSET page_size, Page N = OFFSET (N-1)*page_size. LIMIT and OFFSET are MySQL-specific; other DBs use FETCH FIRST / ROWS ONLY.

QF17. Aggregate Functions (SUM, AVG, MIN, MAX, COUNT)

Foundational
General

PROBLEM STATEMENT

Show the total salary, average salary, minimum salary, maximum salary, and employee count for the entire company.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary

SOLUTION 1: All five aggregates together

```
SELECT COUNT(*) AS total_employees,
       SUM(salary) AS total_salary,
       ROUND(AVG(salary), 2) AS avg_salary,
       MIN(salary) AS min_salary,
       MAX(salary) AS max_salary
FROM Employee;
```

Explanation:
• COUNT(*): total number of rows (employees).
• SUM(salary): sum of all salaries.
• AVG(salary): arithmetic mean. ROUND to 2 decimals.
• MIN/MAX: lowest and highest salaries.

INTUITION: The five standard aggregate functions: COUNT, SUM, AVG, MIN, MAX. Without GROUP BY, they aggregate the entire table into one row.

PATTERN TIP: Aggregates without GROUP BY: one result row for the whole table. Aggregates WITH GROUP BY: one result row per group. ROUND(AVG(), 2) is good practice.

QF18. GROUP BY with HAVING

Foundational
General, TCS

PROBLEM STATEMENT

Find all departments where the average salary is greater than 60000.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary
department	VARCHAR	Department

SOLUTION 1: GROUP BY + HAVING AVG > threshold

```
SELECT department,
       COUNT(*) AS emp_count,
       ROUND(AVG(salary), 2) AS avg_salary
FROM Employee
GROUP BY department
HAVING AVG(salary) > 60000
ORDER BY avg_salary DESC;
```

Explanation:

- GROUP BY department: one group per department.
- AVG(salary) per group: average salary for each dept.
- HAVING AVG(salary) > 60000: filter groups where avg exceeds threshold.
- Cannot use WHERE here because AVG is computed after grouping.

INTUITION: HAVING is WHERE for groups. Use HAVING to filter on aggregated values (COUNT, SUM, AVG, MIN, MAX). Use WHERE to filter individual rows.

PATTERN TIP: Execution order: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY. WHERE cannot see aggregate results; HAVING can.

QF19. INNER JOIN

Foundational
General

PROBLEM STATEMENT

Show each employee name along with their department name (only employees who are assigned to a department).

Table: Employees

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
dept_id	INT	Department ID (FK)

Table: Departments

Column Name	Type	Description
id	INT	Department ID
dept_name	VARCHAR	Department name

SOLUTION 1: INNER JOIN

```
SELECT e.name AS employee_name,
       d.dept_name AS department
FROM Employees e
INNER JOIN Departments d ON e.dept_id = d.id
ORDER BY d.dept_name, e.name;
```

Explanation:

- INNER JOIN: returns only rows where e.dept_id matches d.id in BOTH tables.
- Employees without a matching dept_id are excluded.
- INNER is optional: JOIN alone also means INNER JOIN.

INTUITION: INNER JOIN = intersection. Only rows with matching values in BOTH tables are returned. Employees without a department are excluded.

PATTERN TIP: INNER JOIN = JOIN (identical in MySQL). For clarity in code reviews, write INNER JOIN explicitly. LEFT JOIN includes all from left table; INNER JOIN only matched rows.

QF20. LEFT JOIN

Foundational
General

PROBLEM STATEMENT

Show all employees along with their department name. If an employee has no department, show NULL.

Table: Employees

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
dept_id	INT	Department ID (nullable)

Table: Departments

Column Name	Type	Description
id	INT	Department ID
dept_name	VARCHAR	Department name

SOLUTION 1: LEFT JOIN

```
SELECT e.name AS employee_name,
       d.dept_name AS department
FROM Employees e
LEFT JOIN Departments d ON e.dept_id = d.id
ORDER BY e.name;
```

Explanation:

- LEFT JOIN: returns ALL rows from Employees (left), matching rows from Departments.
- If no department match: dept_name is NULL.
- No employee is excluded, even without a department.

INTUITION: LEFT JOIN = all from left table + matching from right. Missing right-side data shows as NULL. Use when you cannot lose any left-table rows.

PATTERN TIP: LEFT JOIN vs INNER JOIN: LEFT JOIN never loses left-table rows. INNER JOIN only keeps matched rows. When in doubt, ask: 'Can a left-table row have no match?' If yes -> LEFT JOIN.

SECTION 0.3

JOINs and DML

INNER, LEFT, Self-Join, CASE, INSERT, UPDATE, DELETE

QF21. Self-Join

Foundational
JP Morgan, Amazon

PROBLEM STATEMENT

Display each employee alongside their manager name using a self-join.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
managerId	INT	Manager ID (NULL for CEO)

SOLUTION 1: Self-Join with LEFT JOIN for all employees

```
SELECT e.name AS employee,
       COALESCE(m.name, 'No Manager') AS manager
FROM Employee e
```

```
LEFT JOIN Employee m ON e.managerId = m.id
ORDER BY e.name;
```

Explanation:
• Self-join: same table aliased as e (employee) and m (manager).
• LEFT JOIN: include employees without a manager (CEO).
• COALESCE(m.name, 'No Manager'): if NULL (no manager), show 'No Manager'.

INTUITION: Self-join: when a table references itself. Alias the table twice with different names. LEFT JOIN preserves rows without a match (top-level managers with no manager).

PATTERN TIP: COALESCE(expr, default): returns the first non-NULL value. COALESCE(m.name, 'N/A') shows 'N/A' when m.name is NULL. Equivalent to IFNULL in MySQL.

QF22. CASE WHEN Expression

Foundational
General, TCS

PROBLEM STATEMENT

Label each employee salary as Low (< 30000), Medium (30000-70000), or High (> 70000).

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary

SOLUTION 1: CASE WHEN THEN ELSE END

```
SELECT id, name, salary,
       CASE
         WHEN salary < 30000  THEN 'Low'
         WHEN salary <= 70000 THEN 'Medium'
         ELSE 'High'
       END AS salary_band
FROM Employee
ORDER BY salary;
```

Explanation:
• CASE WHEN: SQL's if-then-else. Conditions evaluated top to bottom.
• First matching condition wins.
• ELSE: catch-all for rows not matching any WHEN condition.

INTUITION: CASE WHEN is SQL's conditional logic. Conditions evaluated top-to-bottom - first match wins. ELSE handles everything unmatched.

PATTERN TIP: CASE WHEN can appear in SELECT, WHERE, ORDER BY, GROUP BY, HAVING. It is the universal SQL conditional expression.

QF23. Subquery in WHERE

Foundational
General, Infosys

PROBLEM STATEMENT

Find all employees who earn more than the average company salary.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary

SOLUTION 1: Subquery returning scalar value

```
SELECT id, name, salary
FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee)
ORDER BY salary DESC;
```

Explanation:

- Subquery: (SELECT AVG(salary) FROM Employee) returns one value - the average.
- Outer WHERE: compare each employee's salary to that average.
- This is a scalar subquery - returns exactly one value.

INTUITION: Scalar subquery in WHERE: when you need a computed value to compare against. The subquery runs once, returns one value, and that value is used in the comparison.

PATTERN TIP: Scalar subquery must return exactly one row and one column. If it returns multiple rows, use IN instead of =. Subquery in SELECT: attaches a value to each row.

QF24. String Functions (UPPER, LOWER, CONCAT, LENGTH)

Foundational
General

PROBLEM STATEMENT

Show employee names in uppercase, their email (first_name.last_name@company.com format), and name length.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
first_name	VARCHAR	First name
last_name	VARCHAR	Last name

SOLUTION 1: UPPER, LOWER, CONCAT, LENGTH

```
SELECT id,
```

```
       UPPER(first_name) AS upper_first,
       LOWER(last_name) AS lower_last,
       CONCAT(LOWER(first_name), '.', LOWER(last_name), '@company.com') AS
email,
       LENGTH(first_name) + LENGTH(last_name) AS full_name_length
FROM Employee;
```

Explanation:
• UPPER(): converts to uppercase.
• LOWER(): converts to lowercase.
• CONCAT(a, b, c): joins strings together.
• LENGTH(): number of characters in string.

INTUITION: String functions: UPPER(), LOWER(), CONCAT(), LENGTH(), TRIM(), SUBSTRING(), REPLACE(). These transform string values in SELECT or WHERE.

PATTERN TIP: CONCAT vs ||: MySQL uses CONCAT(). Some DBs use || operator. CONCAT_WS(sep, a, b, c) joins with separator. LENGTH() is byte count; CHAR_LENGTH() is character count.

QF25. Date Functions (YEAR, MONTH, DAY, NOW)

Foundational
Barclays, General

PROBLEM STATEMENT

Find all employees hired in the year 2022, and show how many days they have been employed.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
hire_date	DATE	Hire date

SOLUTION 1: YEAR filter + DATEDIFF

```
SELECT id, name, hire_date,
       DATEDIFF(CURDATE(), hire_date) AS days_employed
FROM Employee
WHERE YEAR(hire_date) = 2022
ORDER BY hire_date;
```

Explanation:
• YEAR(hire_date): extracts year from date. Returns integer.
• DATEDIFF(end, start): number of days between two dates.
• CURDATE(): returns today's date (no time component).

INTUITION: Date extraction functions: YEAR(), MONTH(), DAY(), QUARTER(), WEEKDAY(). DATEDIFF for day differences. CURDATE() for today. NOW() for current timestamp.

PATTERN TIP: Date functions: YEAR(d), MONTH(d)=1-12, DAY(d), QUARTER(d)=1-4, DAYOFWEEK(d)=1-7. CURDATE()=today. NOW()=current timestamp. DATEDIFF(d1,d2)=d1-d2 in days.

QF26. Rank Function (ROW_NUMBER, RANK, DENSE_RANK)

Foundational
JP Morgan, UBS

PROBLEM STATEMENT

Rank all employees by salary (highest first). Show the difference between ROW_NUMBER, RANK, and DENSE_RANK for employees with equal salaries.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary

SOLUTION 1: Three ranking functions side by side

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
       RANK()       OVER (ORDER BY salary DESC) AS rank_val,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank_val
FROM Employee
ORDER BY salary DESC;
```

Explanation:

• ROW_NUMBER: unique sequential number. Ties get different numbers (1,2,3,4).
• RANK: same rank for ties, but skips the next rank (1,1,3).
• DENSE_RANK: same rank for ties, no skipping (1,1,2).
• Example: salaries 100,100,90 -> ROW_NUMBER:1,2,3 | RANK:1,1,3 | DENSE_RANK:1,1,2.

INTUITION: Three ranking functions for ties: ROW_NUMBER (unique, no ties), RANK (ties allowed, gaps after), DENSE_RANK (ties allowed, no gaps). Choose based on requirement.

PATTERN TIP: Quick rule: Nth highest salary -> DENSE_RANK. Unique row position -> ROW_NUMBER. Standard competition ranking (1,1,3) -> RANK.

QF27. Find Employees Without a Department

Foundational
General

PROBLEM STATEMENT

Find all employees who are not assigned to any department.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
dept_id	INT	Department ID (nullable)

SOLUTION 1: IS NULL check

```
SELECT id, name
FROM Employee
WHERE dept_id IS NULL
ORDER BY name;
```

Explanation:

• IS NULL: finds rows where dept_id has no value.
• = NULL would always return no rows in SQL.

INTUITION: Missing/unassigned records: check IS NULL. NULL means the value is absent, unknown, or not applicable. IS NULL is the only way to test for NULL.

PATTERN TIP: Never use = NULL in SQL. Always use IS NULL. To find non-null: IS NOT NULL. To replace NULL in output: COALESCE(col, 'default') or IFNULL(col, 'default').

QF28. UPDATE Statement

Foundational
General

PROBLEM STATEMENT

Give all employees in the Sales department a 10% salary increase.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary
department	VARCHAR	Department

SOLUTION 1: UPDATE with WHERE

```
UPDATE Employee
SET salary = salary * 1.10
WHERE department = 'Sales';

-- Verify the update:
SELECT id, name, salary, department
FROM Employee
WHERE department = 'Sales';
```

Explanation:

• UPDATE table SET col = new_value: modifies existing rows.

• salary * 1.10: 10% increase (multiply by 1.1).
• WHERE department = 'Sales': only update Sales employees.
• Always include WHERE in UPDATE to avoid updating all rows.

INTUITION: UPDATE modifies existing rows. Always use WHERE to limit which rows get updated. Test with SELECT first before running UPDATE in production.

PATTERN TIP: NEVER run UPDATE without WHERE in production - it updates ALL rows. Best practice: run SELECT with the same WHERE first, verify the rows, then run UPDATE.

QF29. DELETE Statement

Foundational
General

PROBLEM STATEMENT

Delete all employees who have been inactive for more than 5 years (hire_date before 2019).

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
hire_date	DATE	Hire date

SOLUTION 1: DELETE with WHERE

```
-- First verify with SELECT:
SELECT id, name, hire_date FROM Employee
WHERE hire_date < '2019-01-01';

-- Then delete:
DELETE FROM Employee
WHERE hire_date < '2019-01-01';
```

Explanation:
• DELETE FROM table WHERE condition: removes matching rows.
• Always test with SELECT first using same WHERE condition.
• hire_date < '2019-01-01': hired before 2019.

INTUITION: DELETE removes rows permanently. Always use WHERE to limit which rows are deleted. Test with SELECT first. Use transactions for safety.

PATTERN TIP: Safety checklist before DELETE: (1) Run SELECT with same WHERE to verify rows. (2) Ensure WHERE is correct. (3) In production, use BEGIN TRANSACTION so you can ROLLBACK if needed.

QF30. INSERT Statement

Foundational

General

PROBLEM STATEMENT

Insert a new employee record into the Employee table.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID (auto-increment)
name	VARCHAR	Employee name
salary	INT	Salary
department	VARCHAR	Department

SOLUTION 1: INSERT INTO with VALUES

```
-- Insert one row:
INSERT INTO Employee (name, salary, department)
VALUES ('Alice Johnson', 75000, 'Engineering');

-- Insert multiple rows:
INSERT INTO Employee (name, salary, department)
VALUES
  ('Bob Smith',   60000, 'HR'),
  ('Carol White', 80000, 'Finance');
```

Explanation:

- INSERT INTO table (col1, col2): specify which columns to insert.
- VALUES (val1, val2): provide values in same order as columns.
- id omitted: auto-increment handles it automatically.
- Multiple rows: separate each set with a comma.

INTUITION: INSERT adds new rows. Always specify column names (not just positions) so the query works even if table columns are reordered.

PATTERN TIP: Always list column names in INSERT. INSERT INTO t VALUES (1,2,3) breaks if columns are added or reordered. INSERT INTO t (col1, col2) VALUES (1,2) is robust.

SECTION 0.4

String, Date and Logic

Functions, COALESCE, UNION, EXISTS vs IN vs JOIN

QF31. CREATE TABLE with Constraints

Foundational
General

PROBLEM STATEMENT

Write the CREATE TABLE statement for an Employee table with proper constraints.

SOLUTION 1: CREATE TABLE with PRIMARY KEY, NOT NULL, DEFAULT, CHECK

```
CREATE TABLE Employee (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(150) UNIQUE NOT NULL,
  salary      DECIMAL(10,2) CHECK (salary > 0),
  department VARCHAR(50) DEFAULT 'General',
  hire_date   DATE NOT NULL DEFAULT (CURDATE()),
  manager_id INT,
  FOREIGN KEY (manager_id) REFERENCES Employee(id)
);
```

Explanation:

- AUTO_INCREMENT: automatically assigns next integer.
- PRIMARY KEY: unique identifier for each row.
- NOT NULL: column cannot be empty.
- UNIQUE: no two rows can have the same value.
- CHECK (salary > 0): salary must be positive.
- DEFAULT: value used when not specified.
- FOREIGN KEY: enforces referential integrity.

INTUITION: Table constraints enforce data integrity. PRIMARY KEY = unique + not null. UNIQUE = no duplicates. NOT NULL = required. CHECK = business rule validation. FOREIGN KEY = referential integrity.

PATTERN TIP: Constraint types: PRIMARY KEY, UNIQUE, NOT NULL, CHECK, DEFAULT, FOREIGN KEY. These prevent bad data from entering the table at the database level.

QF32. ALIASES (Column and Table)

Foundational
General

PROBLEM STATEMENT

Use aliases to make the query output and code more readable.

Table: Employee

Column Name	Type	Description
emp_id	INT	Employee ID
emp_name	VARCHAR	Name
emp_salary	INT	Salary

SOLUTION 1: AS keyword for column and table aliases

```
SELECT
  e.emp_id    AS id,
  e.emp_name  AS name,
  e.emp_salary AS salary,
  e.emp_salary * 1.2 AS salary_after_raise
FROM Employee AS e
WHERE e.emp_salary > 50000;
```

Explanation:
• Column alias: SELECT col AS alias - renames in output.
• Table alias: FROM table AS t - shortens table reference.
• AS is optional: SELECT col alias also works, but AS is clearer.
• Computed column alias: give a name to calculated expressions.

INTUITION: Aliases improve readability. Table aliases are essential for self-joins (Employee e, Employee m). Column aliases rename output headers. AS keyword is optional but recommended.

PATTERN TIP: Column aliases in WHERE: you CANNOT use SELECT aliases in WHERE (executed before SELECT). You CAN use them in ORDER BY and HAVING. To filter on computed values: use a subquery.

QF33. CONCAT and String Operations

Foundational
General

PROBLEM STATEMENT

Create a full name column and format a display string for each employee.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
first_name	VARCHAR	First name
last_name	VARCHAR	Last name
salary	INT	Salary

SOLUTION 1: CONCAT and CONCAT_WS

```
SELECT
  CONCAT(first_name, ' ', last_name) AS full_name,
  CONCAT_WS(', ', last_name, first_name) AS formal_name,
  CONCAT(first_name, ' earns $', FORMAT(salary, 0)) AS display_string
FROM Employee
ORDER BY last_name;
```

Explanation:
• CONCAT(a, b, c): joins strings, returns NULL if any arg is NULL.
• CONCAT_WS(sep, a, b): joins with separator, skips NULL args.
• FORMAT(number, decimals): formats number with commas (1000 -> 1,000).

INTUITION: String manipulation: CONCAT for joining, CONCAT_WS for separator-joined, FORMAT for number display. CONCAT returns NULL if any argument is NULL - use CONCAT_WS or COALESCE to handle NULLs.

PATTERN TIP: CONCAT NULL trap: CONCAT('a', NULL, 'b') = NULL. To handle: CONCAT(COALESCE(col,''), ...) or use CONCAT_WS which skips NULLs automatically.

QF34. SUBSTRING and String Extraction

Foundational
General

PROBLEM STATEMENT

Extract the first 3 characters of each employee name and the domain from their email address.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
email	VARCHAR	Email address

SOLUTION 1: SUBSTRING, LEFT, RIGHT, LOCATE

```
SELECT
  name,
  LEFT(name, 3) AS name_prefix,
  SUBSTRING(email, LOCATE('@', email) + 1) AS email_domain,
  LOCATE('@', email) AS at_position
FROM Employee;
```

Explanation:

• LEFT(str, n): first n characters.
• RIGHT(str, n): last n characters.
• SUBSTRING(str, start, length): extract from position start.
• LOCATE(substr, str): position of first occurrence (1-indexed). Returns 0 if not found.

INTUITION: String extraction: LEFT(), RIGHT(), SUBSTRING(). Find position with LOCATE() or INSTR(). Combine to extract parts like email domain.

PATTERN TIP: SUBSTRING(str, pos, len): pos is 1-indexed. SUBSTRING('Hello', 2, 3) = 'ell'. Without len: SUBSTRING(str, pos) gets from pos to end.

QF35. DATE_FORMAT and Date Display

Foundational
Barclays, General

PROBLEM STATEMENT

Display hire dates in multiple formats and extract date components.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
hire_date	DATE	Hire date

SOLUTION 1: DATE_FORMAT with various format strings

```
SELECT
  name,
  hire_date,
  DATE_FORMAT(hire_date, '%d-%m-%Y') AS dd_mm_yyyy,
  DATE_FORMAT(hire_date, '%M %d, %Y') AS long_format,
  DATE_FORMAT(hire_date, '%Y-%m') AS year_month,
  DAYNAME(hire_date) AS day_of_week,
  MONTHNAME(hire_date) AS month_name
FROM Employee;
```

Explanation:

- DATE_FORMAT(date, format): flexible date formatting.
- %Y=4-digit year, %y=2-digit, %m=month(01-12), %M=month name.
- %d=day(01-31), %D=1st/2nd/3rd, %W=weekday name.
- DAYNAME(): 'Monday'...'Sunday'. MONTHNAME(): 'January'...'December'.

INTUITION: DATE_FORMAT controls how dates are displayed. The format string uses % codes. Most common: '%Y-%m-%d' (ISO), '%d/%m/%Y' (UK), '%m/%d/%Y' (US).

PATTERN TIP: Common DATE_FORMAT codes: %Y=2024, %m=01, %d=15, %H=14, %i=30, %s=00, %p=AM/PM. '%Y-%m-%d %H:%i:%s' = full datetime.

QF36. COALESCE and IFNULL (Handle NULLs)

Foundational
General

PROBLEM STATEMENT

Show employee details with NULL values replaced by default text/values.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
department	VARCHAR	Department (nullable)
bonus	INT	Bonus (nullable)

SOLUTION 1: COALESCE and IFNULL

```
SELECT
  id, name,
  COALESCE(department, 'Unassigned') AS department,
  IFNULL(bonus, 0) AS bonus,
  COALESCE(bonus, salary * 0.05, 0) AS effective_bonus
FROM Employee;
```

Explanation:

- IFNULL(expr, default): returns default if expr is NULL. Two-argument only.
- COALESCE(a, b, c, ...): returns first non-NULL value from the list.
- COALESCE(bonus, salary*0.05, 0): try bonus, then calculated, then 0.

INTUITION: NULL handling: IFNULL for simple two-option replacement. COALESCE for multiple fallback options. Both return the first non-NULL value.

PATTERN TIP: IFNULL(x, y) = COALESCE(x, y) when there are only two options. COALESCE is more flexible (multiple fallbacks) and standard SQL. Prefer COALESCE for portability.

QF37. COUNT with Conditions

Foundational
General, TCS

PROBLEM STATEMENT

Count the total employees and how many are in the IT department, in one query.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
department	VARCHAR	Department

SOLUTION 1: COUNT(*) vs conditional COUNT

```
SELECT
  COUNT(*) AS total_employees,
  COUNT(CASE WHEN department='IT' THEN 1 END) AS it_employees,
  SUM(CASE WHEN department='IT' THEN 1 ELSE 0 END) AS it_employees_v2,
  COUNT(department) AS employees_with_dept
FROM Employee;
```

Explanation:

• COUNT(*): counts all rows.
• COUNT(CASE WHEN condition THEN 1 END): counts rows meeting condition (NULL is not counted).
• SUM(CASE WHEN condition THEN 1 ELSE 0 END): same result, alternative syntax.
• COUNT(department): counts rows where department is NOT NULL.

INTUITION: Conditional count: COUNT(CASE WHEN condition THEN 1 END) counts rows where condition is true. NULL values in COUNT(col) are ignored automatically.

PATTERN TIP: COUNT(*): all rows. COUNT(col): non-NULL values in col. COUNT(CASE WHEN cond THEN 1 END): rows where condition is true. All three are different and have different uses.

QF38. Salary Percentile Ranking

Foundational
JP Morgan, UBS

PROBLEM STATEMENT

For each employee, show their salary rank, what percentage of employees they earn more than, and which quartile they fall in.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary

SOLUTION 1: Multiple ranking window functions

```
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS salary_rank,
       NTILE(4) OVER (ORDER BY salary ASC) AS quartile,
       ROUND(PERCENT_RANK() OVER (ORDER BY salary) * 100, 1) AS percentile
FROM Employee
ORDER BY salary DESC;
```

Explanation:

- RANK(): position (with gaps for ties).
- NTILE(4): divide into 4 quartiles (1=bottom 25%, 4=top 25%).
- PERCENT_RANK(): (rank-1)/(N-1), values 0.0 to 1.0.
- NTILE(4) ORDER BY salary ASC: quartile 4 = highest earners.

INTUITION: Distribution functions: NTILE(n) divides into n buckets. PERCENT_RANK gives 0-1 score. Combine with CASE to create categories (Top/Mid/Low).

PATTERN TIP: NTILE(4) ORDER BY salary ASC: bucket 1 = bottom 25%, bucket 4 = top 25%. PERCENT_RANK >= 0.75 = top 25%. Both methods, choose based on preference.

QF39. UNION and UNION ALL

Foundational
General

PROBLEM STATEMENT

Combine active and inactive employee lists into one result. Show which list each employee came from.

Table: ActiveEmployees

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary

Table: InactiveEmployees

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary

SOLUTION 1: UNION ALL with status column

```
SELECT id, name, salary, 'Active' AS status
FROM ActiveEmployees
UNION ALL
SELECT id, name, salary, 'Inactive' AS status
FROM InactiveEmployees
ORDER BY status, name;
```

Explanation:

- UNION ALL: combines both result sets, keeps ALL rows including duplicates.
- UNION: combines and removes duplicates (slower).
- Both queries must have same number of columns with compatible types.
- ORDER BY at the end applies to the combined result.

INTUITION: UNION: combine results from two queries. UNION ALL is faster (no deduplication). Use UNION only when you need to remove duplicates across both sets.

PATTERN TIP: UNION removes duplicates (runs DISTINCT). UNION ALL keeps everything (faster). Column count must match. Data types must be compatible. ORDER BY goes at the very end.

QF40. EXISTS vs IN vs JOIN

Foundational
JP Morgan, General

PROBLEM STATEMENT

Find all departments that have at least one employee earning more than 100000. Show three different approaches.

Table: Department

Column Name	Type	Description
id	INT	Dept ID
name	VARCHAR	Dept name

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary
dept_id	INT	FK to Department

SOLUTION 1: EXISTS

```
SELECT d.name
FROM Department d
WHERE EXISTS (
  SELECT 1 FROM Employee e
  WHERE e.dept_id = d.id AND e.salary > 100000
);
```

Explanation:
• EXISTS: returns TRUE if subquery returns ANY row.
• SELECT 1: we don't care about the value, just whether a row exists.
• Stops at first matching row - efficient for large tables.

SOLUTION 2: IN with Subquery

```
SELECT name FROM Department
WHERE id IN (
  SELECT DISTINCT dept_id FROM Employee
  WHERE salary > 100000
);
```

Explanation:
• Subquery returns list of dept_ids with high-earning employees.
• IN matches department id against that list.

SOLUTION 3: JOIN + DISTINCT

```
SELECT DISTINCT d.name
FROM Department d
JOIN Employee e ON d.id = e.dept_id
WHERE e.salary > 100000;
```

Explanation:
• JOIN: matches departments to their high-earning employees.
• DISTINCT: a department may have multiple high earners, show it once.

INTUITION: Three ways to find 'has at least one matching record': EXISTS (most readable), IN (subquery), JOIN + DISTINCT. EXISTS is often most efficient as it short-circuits on first match.

PATTERN TIP: EXISTS vs IN: EXISTS is generally faster for large subqueries (stops at first match). IN loads all values into memory. Both are correct. Use EXISTS for clarity.

SECTION 0.5

Window Functions and Mixed

LAG/LEAD, Running Total, Top-N, Pivot, Median, Full Report

QF41. Window Function LAG and LEAD

Foundational
JP Morgan, Barclays

PROBLEM STATEMENT

For each monthly sale, show the previous month sale and the next month sale.

Table: MonthlySales

Column Name	Type	Description

sale_month	DATE	Month (YYYY-MM-01)
amount	DECIMAL	Sales amount

SOLUTION 1: LAG and LEAD

```
SELECT
  sale_month,
  amount AS current_sales,
  LAG(amount, 1, 0) OVER (ORDER BY sale_month) AS prev_month_sales,
  LEAD(amount, 1, 0) OVER (ORDER BY sale_month) AS next_month_sales,
  amount - LAG(amount, 1) OVER (ORDER BY sale_month) AS mom_change
FROM MonthlySales
ORDER BY sale_month;
```

Explanation:

- LAG(col, n, default): value n rows BEFORE current row. Default if no prior row.
- LEAD(col, n, default): value n rows AFTER current row. Default if no next row.
- Both use OVER(ORDER BY) to define the sequence.
- amount - LAG(amount): month-over-month change.

INTUITION: LAG/LEAD: access values from previous/next rows without a self-join. LAG looks back, LEAD looks forward. The third argument is the default value when no row exists.

PATTERN TIP: LAG(col, 1): one row back. LAG(col, 2): two rows back. LAG(col, 1, 0): default 0 for first row. LEAD is the forward mirror of LAG.

QF42. Cumulative Sum with Window Function

Foundational
General

PROBLEM STATEMENT

Show running total of sales month by month.

Table: MonthlySales

Column Name	Type	Description
sale_month	DATE	Month
amount	DECIMAL	Monthly sales

SOLUTION 1: SUM() OVER (ORDER BY)

```
SELECT
  sale_month,
  amount AS monthly_sales,
  SUM(amount) OVER (ORDER BY sale_month) AS running_total,
  ROUND(AVG(amount) OVER (ORDER BY sale_month), 2) AS running_avg
FROM MonthlySales
ORDER BY sale_month;
```

Explanation:

- SUM OVER (ORDER BY): cumulative sum from first row to current.
- Default frame: UNBOUNDED PRECEDING to CURRENT ROW.
- Running AVG: average of all rows up to current.

INTUITION: Running total = SUM() OVER (ORDER BY). The window grows from the first row to the current row, accumulating the sum.

PATTERN TIP: Running SUM without PARTITION BY: single running total for the whole result. With PARTITION BY group: running total resets for each group.

QF43. Find Top N per Group

Foundational
JP Morgan, Amazon

PROBLEM STATEMENT

Find the top 2 highest-paid employees in each department.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary
department	VARCHAR	Department

SOLUTION 1: ROW_NUMBER PARTITION BY department

```
SELECT department, name, salary
FROM (
  SELECT department, name, salary,
         ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn
  FROM Employee
) ranked
WHERE rn <= 2
ORDER BY department, salary DESC;
```

Explanation:

- PARTITION BY department: row number resets for each department.
- ORDER BY salary DESC: highest salary = rn 1.
- WHERE rn <= 2: top 2 per department.

INTUITION: Top-N per group: ROW_NUMBER PARTITION BY group ORDER BY metric DESC, then WHERE rn <= N. PARTITION BY resets the ranking for each group.

PATTERN TIP: ROW_NUMBER: exactly top-N even with ties. DENSE_RANK: may return more than N when there are ties at the boundary. Choose based on whether ties should be included.

QF44. Pivot Data with CASE WHEN

Foundational
Barclays, JP Morgan

PROBLEM STATEMENT

Show count of employees per department in a single row (pivot format): one column per department.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
department	VARCHAR	Department (IT/HR/Finance/Sales)

SOLUTION 1: Manual pivot with SUM CASE

```
SELECT
  SUM(CASE WHEN department='IT'      THEN 1 ELSE 0 END) AS IT_count,
  SUM(CASE WHEN department='HR'      THEN 1 ELSE 0 END) AS HR_count,
  SUM(CASE WHEN department='Finance' THEN 1 ELSE 0 END) AS Finance_count,
  SUM(CASE WHEN department='Sales'   THEN 1 ELSE 0 END) AS Sales_count,
  COUNT(*) AS total
FROM Employee;
```

Explanation:

- SUM(CASE WHEN dept='X' THEN 1 ELSE 0 END): count employees in dept X.
- One expression per department = one column per department.
- This is called a PIVOT - rows converted to columns.

INTUITION: PIVOT in MySQL = SUM(CASE WHEN category='X' THEN 1 ELSE 0 END) for each category. Creates one column per category value. MySQL has no native PIVOT operator.

PATTERN TIP: SUM(CASE WHEN cond THEN 1 ELSE 0 END) = COUNT of rows meeting condition. This is the most versatile conditional aggregation pattern in SQL.

QF45. Recursive Thinking: All Managers of an Employee

Foundational
JP Morgan

PROBLEM STATEMENT

For a given employee, list all of their managers up the hierarchy (direct manager, their manager, etc). Use a fixed-level JOIN approach for up to 3 levels.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
managerId	INT	Manager ID

SOLUTION 1: Multi-level self-join (up to 3 levels)

```
SELECT
  e.name AS employee,
  m1.name AS direct_manager,
  m2.name AS manager_level_2,
  m3.name AS manager_level_3
```

```
FROM Employee e
LEFT JOIN Employee m1 ON e.managerId = m1.id
LEFT JOIN Employee m2 ON m1.managerId = m2.id
LEFT JOIN Employee m3 ON m2.managerId = m3.id
WHERE e.id = 5;  -- Replace 5 with the employee id
```

Explanation:
• Each LEFT JOIN goes one level up the hierarchy.
• LEFT JOIN: if a level doesn't exist, NULL is shown.
• For true unlimited recursion, MySQL 8.0+ supports WITH RECURSIVE.

INTUITION: Hierarchy traversal without recursion: chain self-joins, one per level. LEFT JOIN so levels that don't exist show NULL. True recursive hierarchy needs WITH RECURSIVE CTE (MySQL 8.0+).

PATTERN TIP: For fixed-depth hierarchies: chain LEFT JOINs. For variable depth: use WITH RECURSIVE. In interviews, showing you know both approaches demonstrates depth of knowledge.

QF46. Find Gaps in Data (Missing Dates)

Foundational
Barclays, General

PROBLEM STATEMENT

Find all dates in January 2024 where no sales were recorded.

Table: Sales

Column Name	Type	Description
sale_date	DATE	Date of sale
amount	DECIMAL	Sales amount

SOLUTION 1: Generate dates + LEFT JOIN IS NULL

```
-- Generate all Jan 2024 dates using a numbers table trick
SELECT DATE_ADD('2024-01-01', INTERVAL n DAY) AS missing_date
FROM (
  SELECT 0 AS n UNION SELECT 1 UNION SELECT 2 UNION SELECT 3
  UNION SELECT 4 UNION SELECT 5 UNION SELECT 6 UNION SELECT 7
  UNION SELECT 8 UNION SELECT 9 UNION SELECT 10 UNION SELECT 11
  UNION SELECT 12 UNION SELECT 13 UNION SELECT 14 UNION SELECT 15
  UNION SELECT 16 UNION SELECT 17 UNION SELECT 18 UNION SELECT 19
  UNION SELECT 20 UNION SELECT 21 UNION SELECT 22 UNION SELECT 23
  UNION SELECT 24 UNION SELECT 25 UNION SELECT 26 UNION SELECT 27
  UNION SELECT 28 UNION SELECT 29 UNION SELECT 30
) numbers
WHERE DATE_ADD('2024-01-01', INTERVAL n DAY) <= '2024-01-31'
  AND DATE_ADD('2024-01-01', INTERVAL n DAY) NOT IN (
    SELECT DISTINCT sale_date FROM Sales
    WHERE sale_date BETWEEN '2024-01-01' AND '2024-01-31'
  );
```

Explanation:
• Generate sequence: UNION of integers 0-30.
• DATE_ADD: converts numbers to dates.

• NOT IN: dates not in the sales table = missing dates.

INTUITION: Finding missing dates: generate all expected dates, then find which ones don't exist in the data. Generate a number sequence, convert to dates, then anti-join.

PATTERN TIP: Date gap finding requires generating expected dates first. For production, a Calendar table (pre-populated with all dates) is preferred over generating inline.

QF47. String Aggregation (GROUP_CONCAT)

Foundational
LeetCode, Barclays

PROBLEM STATEMENT

For each department, list all employee names as a comma-separated string.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
department	VARCHAR	Department

SOLUTION 1: GROUP_CONCAT

```
SELECT department,
       COUNT(*) AS emp_count,
       GROUP_CONCAT(name ORDER BY name SEPARATOR ', ') AS employee_names
FROM Employee
GROUP BY department
ORDER BY department;
```

Explanation:

• GROUP_CONCAT: aggregates strings within a group.
• ORDER BY name: alphabetical order within the list.
• SEPARATOR ', ': use comma-space as separator.
• Default separator is comma.

INTUITION: GROUP_CONCAT is MySQL's string aggregation function. It concatenates column values within each group. Essential for creating comma-separated lists from rows.

PATTERN TIP: GROUP_CONCAT options: GROUP_CONCAT([DISTINCT] col [ORDER BY col] [SEPARATOR 'sep']). Default separator is comma. MySQL has a max length limit (default 1024 bytes) - increase with group_concat_max_len.

QF48. Conditional Update with CASE

Foundational
General

PROBLEM STATEMENT

Give different salary increases based on department: IT gets 15%, HR gets 10%, all others get 5%.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary
department	VARCHAR	Department

SOLUTION 1: UPDATE with CASE WHEN

```
UPDATE Employee
SET salary = salary * CASE
  WHEN department = 'IT' THEN 1.15
  WHEN department = 'HR' THEN 1.10
  ELSE 1.05
END;

-- Verify:
SELECT name, department, salary FROM Employee ORDER BY department;
```

Explanation:

- SET salary = salary * CASE: multiply existing salary by the multiplier.
- CASE in SET: different multiplier per department.
- No WHERE clause: updates ALL employees.

INTUITION: Conditional UPDATE: use CASE WHEN in the SET clause to apply different values to different rows in one UPDATE statement. More efficient than multiple UPDATE statements.

PATTERN TIP: CASE in UPDATE SET: calculates per-row value dynamically. More efficient than multiple UPDATE...WHERE statements. Always verify with SELECT before running the UPDATE.

QF49. Finding Median without Built-in Function

Foundational
JP Morgan, UBS

PROBLEM STATEMENT

Find the median salary of all employees. Handle both odd and even counts.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
salary	INT	Salary

SOLUTION 1: Counting method

```
SELECT AVG(salary) AS median_salary
FROM (
```

```
  SELECT salary
  FROM Employee
  ORDER BY salary
  LIMIT 2 - (SELECT COUNT(*) FROM Employee) % 2
  OFFSET FLOOR(((SELECT COUNT(*) FROM Employee) - 1) / 2)
) AS middle_rows;
```

Explanation:

- COUNT(*) % 2: 0 if even, 1 if odd number of employees.
- LIMIT 2 - (count%2): LIMIT 1 for odd count, LIMIT 2 for even count.
- OFFSET FLOOR((count-1)/2): skip to the middle row(s).
- AVG: for odd, one row AVG = itself. For even, average of two middle rows.

INTUITION: Median requires finding the middle row(s). For odd N: one middle row. For even N: average two middle rows. OFFSET positions to the middle, LIMIT takes 1 or 2 rows.

PATTERN TIP: Median in MySQL 8.0+: use PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) if available. Otherwise, the OFFSET/LIMIT approach or ROW_NUMBER approach both work.

QF50. Complete Employee Report (Multi-concept)

Foundational
JP Morgan, Barclays, UBS

PROBLEM STATEMENT

Write a comprehensive query: show each employee with their department, manager name, salary rank within department, whether they earn above department average, and their salary band.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary
dept_id	INT	Department ID
managerId	INT	Manager ID (nullable)

SOLUTION 1: Full combined query with JOIN, Window Functions, CASE

```
SELECT
  e.id,
  e.name AS employee_name,
  COALESCE(m.name, 'Top Level') AS manager_name,
  e.salary,
  DENSE_RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS
dept_salary_rank,
  CASE
    WHEN e.salary > AVG(e.salary) OVER (PARTITION BY e.dept_id) THEN 'Above Avg'
    WHEN e.salary = AVG(e.salary) OVER (PARTITION BY e.dept_id) THEN 'At Avg'
    ELSE 'Below Avg'
  END AS vs_dept_avg,
  CASE
    WHEN e.salary < 30000 THEN 'Low'
```

```
    WHEN e.salary <= 70000 THEN 'Medium'
    ELSE 'High'
  END AS salary_band
FROM Employee e
LEFT JOIN Employee m ON e.managerId = m.id
ORDER BY e.dept_id, e.salary DESC;
```

Explanation:
- LEFT JOIN Employee m: self-join to get manager name.
- COALESCE: replace NULL manager with 'Top Level'.
- DENSE_RANK PARTITION BY dept_id: rank within each department.
- AVG OVER PARTITION BY: department average attached to every row.
- CASE for vs_dept_avg: compare individual to department average.
- CASE for salary_band: salary categorisation.

INTUITION: Real-world queries combine multiple concepts: self-join for hierarchy, window functions for ranking and aggregates, CASE for categorisation, COALESCE for NULL handling.

PATTERN TIP: When answering complex interview questions, build incrementally: (1) Start with the base table and JOINs, (2) Add window functions, (3) Add CASE expressions, (4) Add WHERE/ORDER BY. Test each step.

SECTION 1

Basic SELECT and Classic Interview Patterns

Q1. Customer with Highest Tip

Easy
Barclays

PROBLEM STATEMENT

Find the customer_id of the customer whose last name is "kate" (case-insensitive). If multiple, return the one with the highest tip.

Table: customer_review

Column Name	Type	Description
customer_id	INT	Unique customer ID
customer_name	VARCHAR	Full name of customer
tip_amount	INT	Tip paid
comments	VARCHAR	Experience comment

SOLUTION 1: WHERE LOWER + ORDER BY LIMIT 1

```
SELECT customer_id
FROM customer_review
WHERE LOWER(customer_name) LIKE '% kate'
   OR LOWER(customer_name) = 'kate'
ORDER BY tip_amount DESC
LIMIT 1;
```

Explanation:
• LOWER(): converts name to lowercase for case-insensitive match.
• LIKE '% kate': last word is 'kate' (space + kate).
• ORDER BY tip_amount DESC: highest tipper first.
• LIMIT 1: return only top row.

SOLUTION 2: Window RANK()

```
SELECT customer_id FROM (
  SELECT customer_id,
         RANK() OVER (ORDER BY tip_amount DESC) AS rnk
  FROM customer_review
  WHERE LOWER(customer_name) LIKE '% kate'
     OR LOWER(customer_name) = 'kate'
) t WHERE rnk = 1;
```

Explanation:
• RANK() = 1: highest tip. Ties both get rank 1.

INTUITION: 'Case-insensitive' = LOWER(). 'Last name is X' = LIKE '% X'. Pick one row by max value = ORDER BY DESC LIMIT 1 or RANK()=1.

PATTERN TIP: LIKE '% kate' matches any name ending in ' kate'. LOWER() ensures true case-insensitivity.

Q2. Monthly Sales Calculation

Easy
Barclays

PROBLEM STATEMENT

Determine monthly sales ordered by month. Daily sale = cost x volume.

Table: coffee

Column Name	Type	Description
cost	INT	Cost for that day
dates	DATE	Date of sale
volume	INT	Volume sold

SOLUTION 1: MONTH() + GROUP BY + SUM

```
SELECT MONTH(dates) AS month,
       SUM(cost * volume) AS monthly_sales
FROM coffee
GROUP BY MONTH(dates)
ORDER BY month;
```

Explanation:
• MONTH(dates): extracts month number 1-12.
• cost * volume: daily sale.
• SUM: aggregates across all days in the month.
• GROUP BY MONTH: one row per month.

SOLUTION 2: DATE_FORMAT for multi-year data

```
SELECT DATE_FORMAT(dates,'%Y-%m') AS year_month,
       SUM(cost * volume) AS monthly_sales
FROM coffee
GROUP BY DATE_FORMAT(dates,'%Y-%m')
ORDER BY year_month;
```

Explanation:

• DATE_FORMAT '%Y-%m': groups as '2024-01' so Jan-2023 != Jan-2024.

INTUITION: Monthly aggregation: extract month, compute row metric, SUM and GROUP BY. Multi-year: use DATE_FORMAT('%Y-%m').

PATTERN TIP: MONTH() alone merges months across years. DATE_FORMAT('%Y-%m') is the safe multi-year option.

Q3. October Month Data

Easy
Barclays

PROBLEM STATEMENT

Find all records from October (irrespective of year), sorted ascending by Customer_ID.

Table: bank

Column Name	Type	Description
Info_Date	DATE	Date info received
Customer_ID	INT	Customer ID
Transactions	INT	Number of transactions
Amount	INT	Account amount

SOLUTION 1: MONTH() = 10

```
SELECT *
FROM bank
WHERE MONTH(Info_Date) = 10
ORDER BY Customer_ID ASC;
```

Explanation:

• MONTH(Info_Date) = 10: October = 10.
• No YEAR() filter: 'irrespective of year'.
• ORDER BY ASC: ascending as required.

SOLUTION 2: EXTRACT alternative

```
SELECT *
FROM bank
WHERE EXTRACT(MONTH FROM Info_Date) = 10
ORDER BY Customer_ID ASC;
```

Explanation:

• EXTRACT(MONTH FROM date): ANSI SQL standard. Same result as MONTH().

INTUITION: 'Irrespective of year' = use MONTH() only, no YEAR() filter. October = 10.

PATTERN TIP: Month numbers: Jan=1, Mar=3, Oct=10, Dec=12. MONTHNAME() returns string - less efficient for filtering.

Q4. Insurance Claim Eligibility

Medium
Barclays

PROBLEM STATEMENT

An eligible customer receives 150% of claimed amount. All conditions must hold: Life Insurance, no injury reported, major severity, employed, claim < 30 x premium. Return eligible customers and final claim amount.

Table: insurance

Column Name	Type	Description
customer_id	INT	Customer ID
insurance_type	VARCHAR	Type of insurance
injury_reported	VARCHAR	Yes or No
incident_severity	VARCHAR	minor or major
employment_status	VARCHAR	employed or unemployed
claim_amount	DECIMAL	Claimed amount
premium_amount	DECIMAL	Monthly premium

SOLUTION 1: Multi-condition WHERE + computed column

```
SELECT customer_id,
       claim_amount * 1.5 AS final_claim_amount
FROM insurance
WHERE insurance_type = 'Life Insurance'
  AND injury_reported = 'No'
  AND incident_severity = 'major'
  AND employment_status = 'employed'
  AND claim_amount < 30 * premium_amount;
```

Explanation:

• claim_amount * 1.5: 150% payout.
• All conditions chained with AND: all must be true.
• claim_amount < 30 * premium_amount: compares two columns directly.

INTUITION: Multi-rule eligibility: each business rule becomes a WHERE AND condition. 'All of the following' = AND. 150% = multiply by 1.5.

PATTERN TIP: 'All conditions' = AND. 'Any condition' = OR. Direct column comparison in WHERE: col1 < 30 * col2 is valid.

Q5. Second Highest Salary

Medium
JP Morgan, Barclays, UBS

PROBLEM STATEMENT

Find the second highest distinct salary. Return NULL if it does not exist.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
salary	INT	Salary

SOLUTION 1: Subquery + MAX

```
SELECT MAX(salary) AS SecondHighestSalary
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Explanation:

- Inner: gets max salary.
- Outer WHERE < max: excludes top salary.
- Outer MAX: highest among remaining = 2nd highest.
- Returns NULL automatically if empty.

SOLUTION 2: LIMIT OFFSET

```
SELECT DISTINCT salary AS SecondHighestSalary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Explanation:

- DISTINCT: remove duplicate salaries.
- ORDER BY DESC: highest first.
- LIMIT 1 OFFSET 1: skip 1, take next.

SOLUTION 3: DENSE_RANK

```
SELECT salary AS SecondHighestSalary
FROM (
  SELECT salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM Employee
) t
WHERE rnk = 2
LIMIT 1;
```

Explanation:

- DENSE_RANK: no gaps in ranking. 1,1,2 not 1,1,3.
- rnk = 2: exactly the 2nd distinct salary.

INTUITION: 2nd highest = exclude MAX then find MAX again. Or DENSE_RANK = 2. DISTINCT is critical when multiple employees share salaries.

PATTERN TIP: DENSE_RANK: ties get same rank, next is consecutive (1,1,2). RANK: ties get same rank, gap after (1,1,3). For Nth highest use DENSE_RANK = N or LIMIT 1 OFFSET N-1.

Q6. Nth Highest Salary

Medium
JP Morgan, UBS

PROBLEM STATEMENT

Return the Nth highest distinct salary. Return NULL if there is no Nth highest.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
salary	INT	Salary

SOLUTION 1: LIMIT OFFSET (N-1)

```
-- For N=3 (3rd highest):
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 2;  -- OFFSET = N-1
```

Explanation:

• DISTINCT: treat equal salaries as one rank.
• OFFSET N-1: skip N-1 rows. N=1 skip 0, N=3 skip 2.

SOLUTION 2: DENSE_RANK = N

```
SELECT salary
FROM (
  SELECT salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM Employee
) t
WHERE rnk = 3  -- replace 3 with N
LIMIT 1;
```

Explanation:

• DENSE_RANK assigns continuous ranks without gaps.
• WHERE rnk = N: the Nth distinct salary.

INTUITION: Nth highest generalises 2nd highest. OFFSET N-1 skips exactly N-1 rows. DENSE_RANK = N is direct.

PATTERN TIP: LIMIT M OFFSET N: skip N rows, take M. 3rd highest: LIMIT 1 OFFSET 2.

Q7. Employees Earning More Than Their Manager

Medium
JP Morgan, Barclays

PROBLEM STATEMENT
Find names of employees who earn more than their direct manager.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary
managerId	INT	Manager ID (references id)

SOLUTION 1: Self-Join

```
SELECT e.name AS Employee
FROM Employee e
JOIN Employee m ON e.managerId = m.id
WHERE e.salary > m.salary;
```

Explanation:
• Self-join: same table aliased as e (employee) and m (manager).
• e.managerId = m.id: links employee to their manager row.
• WHERE e.salary > m.salary: employee earns more.

SOLUTION 2: Correlated Subquery

```
SELECT name AS Employee
FROM Employee e1
WHERE salary > (
  SELECT salary FROM Employee e2
  WHERE e2.id = e1.managerId
);
```

Explanation:
• For each employee row, fetch their manager's salary via subquery.
• Compare employee salary to manager salary.

INTUITION: Self-reference (managerId -> id in same table) = SELF-JOIN. Alias table twice: employee alias and manager alias.

PATTERN TIP: Self-join: JOIN same_table alias2 ON alias1.fk = alias2.pk. Meaningful aliases (e, m) make WHERE readable.

Q8. Duplicate Emails

Easy
JP Morgan, LeetCode

PROBLEM STATEMENT
Find all emails that appear more than once in the Person table.

Table: Person

Column Name	Type	Description
id	INT	Person ID
email	VARCHAR	Email address

SOLUTION 1: GROUP BY + HAVING COUNT > 1

```
SELECT email
FROM Person
GROUP BY email
HAVING COUNT(*) > 1;
```

Explanation:

• GROUP BY email: groups rows by email value.
• HAVING COUNT(*) > 1: only emails appearing more than once.
• HAVING filters after aggregation - cannot use WHERE here.

INTUITION: Duplicates = GROUP BY the column, HAVING COUNT > 1. HAVING filters groups after aggregation.

PATTERN TIP: WHERE filters rows (before GROUP BY). HAVING filters groups (after GROUP BY). This is the single most important WHERE vs HAVING distinction.

Q9. Customers Who Never Ordered

Easy
JP Morgan, LeetCode

PROBLEM STATEMENT

Find all customers who have never placed an order.

Table: Customers

Column Name	Type	Description
id	INT	Customer ID
name	VARCHAR	Name

Table: Orders

Column Name	Type	Description
id	INT	Order ID
customerId	INT	FK to Customers

SOLUTION 1: LEFT JOIN + IS NULL

```
SELECT c.name AS Customers
FROM Customers c
LEFT JOIN Orders o ON c.id = o.customerId
WHERE o.id IS NULL;
```

Explanation:

• LEFT JOIN: all customers, NULLs where no order exists.

• WHERE o.id IS NULL: customers with no order.

SOLUTION 2: NOT IN Subquery

```
SELECT name AS Customers
FROM Customers
WHERE id NOT IN (
  SELECT customerId FROM Orders
);
```

Explanation:

• Subquery: all customerIds who have orders.
• NOT IN: excludes those customers.
• WARNING: NOT IN fails if subquery contains NULL values.

INTUITION: No match in another table = LEFT JOIN IS NULL, NOT IN, or NOT EXISTS. LEFT JOIN IS NULL is safest when NULLs may exist.

PATTERN TIP: NOT IN with NULL in subquery returns no rows. LEFT JOIN IS NULL handles NULLs correctly. Prefer LEFT JOIN IS NULL.

Q10. Rank Scores (No Gaps in Ranking)

Medium
LeetCode, UBS

PROBLEM STATEMENT

Rank scores highest to lowest. Ties get same rank. After a tie, ranking is consecutive (no gaps). Example: scores 100,100,90 -> ranks 1,1,2.

Table: Scores

Column Name	Type	Description
id	INT	Score ID
score	DECIMAL	Score value

SOLUTION 1: DENSE_RANK()

```
SELECT score,
       DENSE_RANK() OVER (ORDER BY score DESC) AS `rank`
FROM Scores
ORDER BY score DESC;
```

Explanation:

• DENSE_RANK: ties get same rank, next rank is consecutive (no gap).
• ORDER BY score DESC: highest score = rank 1.
• Backtick around 'rank': it is a MySQL reserved keyword.

INTUITION: No-gap ranking = DENSE_RANK. Gap-allowed = RANK. Unique sequential = ROW_NUMBER. Memorise these three.

PATTERN TIP: ROW_NUMBER: unique (1,2,3,4). RANK: gap after tie (1,1,3). DENSE_RANK: no gap (1,1,2).

SECTION 2

JOINs — INNER, LEFT, SELF, CROSS

Q11. Combine Two Tables (LEFT JOIN)

Easy
LeetCode

PROBLEM STATEMENT

Report first name, last name, city, state for each person. If no address exists, report NULL for city and state.

Table: Person

Column Name	Type	Description
personId	INT	Person ID
lastName	VARCHAR	Last name
firstName	VARCHAR	First name

Table: Address

Column Name	Type	Description
addressId	INT	Address ID
personId	INT	FK to Person
city	VARCHAR	City
state	VARCHAR	State

SOLUTION 1: LEFT JOIN

```
SELECT p.firstName, p.lastName, a.city, a.state
FROM Person p
LEFT JOIN Address a ON p.personId = a.personId;
```

Explanation:

- LEFT JOIN: all rows from Person, matching from Address.
- No address match: city and state are NULL as required.

INTUITION: LEFT JOIN = all from left table + matching from right. Missing right-side data becomes NULL.

PATTERN TIP: INNER JOIN = only matching rows in both. LEFT JOIN = all left rows, NULLs for missing right data.

Q12. Rising Temperature (Self-Join on Date)

Medium
LeetCode, JP Morgan

PROBLEM STATEMENT

Find record ids where temperature was higher than the previous day.

Table: Weather

Column Name	Type	Description
id	INT	Record ID
recordDate	DATE	Date
temperature	INT	Temperature

SOLUTION 1: Self-Join with DATEDIFF

```
SELECT w1.id
FROM Weather w1
JOIN Weather w2
  ON DATEDIFF(w1.recordDate, w2.recordDate) = 1
WHERE w1.temperature > w2.temperature;
```

Explanation:

- Self-join: w1 = today, w2 = yesterday.
- DATEDIFF = 1: w1 is exactly 1 day after w2.
- WHERE: today warmer than yesterday.

SOLUTION 2: LAG Window Function

```
SELECT id
FROM (
  SELECT id, temperature,
         LAG(temperature) OVER (ORDER BY recordDate) AS prev_temp
  FROM Weather
) t
WHERE temperature > prev_temp;
```

Explanation:

- LAG(temperature): previous row value.
- WHERE temperature > prev_temp: warmer than yesterday.

INTUITION: Compare row to previous row: self-join with DATEDIFF=1, or LAG() window function. LAG is the modern cleaner approach.

PATTERN TIP: LAG(col, N): value N rows back. LEAD(col, N): value N rows ahead. Both use OVER(ORDER BY ...).

Q13. Department Highest Salary

Medium
LeetCode, JP Morgan

PROBLEM STATEMENT

Find employees who have the highest salary in each department. Include ties.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary
departmentId	INT	FK to Department

Table: Department

Column Name	Type	Description
id	INT	Dept ID
name	VARCHAR	Dept name

SOLUTION 1: Subquery Row-Value IN

```
SELECT d.name AS Department, e.name AS Employee, e.salary
FROM Employee e
JOIN Department d ON e.departmentId = d.id
WHERE (e.departmentId, e.salary) IN (
  SELECT departmentId, MAX(salary)
  FROM Employee
  GROUP BY departmentId
);
```

Explanation:

• Subquery: MAX salary per department.
• Row-value IN: both departmentId AND salary must match.
• Ties: multiple employees at same max salary are all returned.

SOLUTION 2: DENSE_RANK PARTITION BY

```
SELECT Department, Employee, Salary
FROM (
  SELECT d.name AS Department, e.name AS Employee, e.salary AS Salary,
         DENSE_RANK() OVER (PARTITION BY e.departmentId ORDER BY e.salary DESC)
AS rnk
  FROM Employee e
  JOIN Department d ON e.departmentId = d.id
) t
WHERE rnk = 1;
```

Explanation:

• PARTITION BY departmentId: rank resets per department.
• DENSE_RANK = 1: highest salary(s) per department.

INTUITION: Max-per-group with ties: row-value subquery or DENSE_RANK PARTITION BY. PARTITION BY divides dataset into independent groups.

PATTERN TIP: PARTITION BY in window functions = per-group context. Rank resets for each partition. Like GROUP BY but for windows.

Q14. Consecutive Numbers (Three in a Row)

Hard
LeetCode, UBS

PROBLEM STATEMENT

Find all numbers that appear at least three times consecutively.

Table: Logs

Column Name	Type	Description
id	INT	Sequential ID
num	INT	Number logged

SOLUTION 1: Self-Join Three Times

```
SELECT DISTINCT l1.num AS ConsecutiveNums
FROM Logs l1
JOIN Logs l2 ON l1.id = l2.id - 1
JOIN Logs l3 ON l1.id = l3.id - 2
WHERE l1.num = l2.num AND l2.num = l3.num;
```

Explanation:

- l2.id = l1.id + 1: consecutive.
- l3.id = l1.id + 2: two ahead.
- WHERE all three equal: same number 3 rows in a row.
- DISTINCT: may appear in multiple groups.

SOLUTION 2: LAG + LEAD

```
SELECT DISTINCT num AS ConsecutiveNums
FROM (
  SELECT num,
         LAG(num)  OVER (ORDER BY id) AS prev1,
         LEAD(num) OVER (ORDER BY id) AS next1
  FROM Logs
) t
WHERE num = prev1 AND num = next1;
```

Explanation:

- LAG: row before. LEAD: row after.
- WHERE all three equal: 3 in a row.

INTUITION: 3 consecutive same values: self-join on id offsets (id, id-1, id-2), or LAG/LEAD. Consecutive = IDs differ by exactly 1.

PATTERN TIP: LAG/LEAD safer than self-join when IDs have gaps. Window ordering handles gaps automatically.

Q15. Delete Duplicate Emails (Keep Lowest ID)

Medium
LeetCode

PROBLEM STATEMENT

DELETE all duplicate emails, keeping only the row with the smallest id.

Table: Person

Column Name	Type	Description
id	INT	Primary key
email	VARCHAR	Email

SOLUTION 1: Self-Join DELETE

```
DELETE p1
FROM Person p1
JOIN Person p2
  ON p1.email = p2.email
  AND p1.id > p2.id;
```

Explanation:

- DELETE p1: deletes rows from alias p1.
- Self-join: pairs rows with same email.
- p1.id > p2.id: delete higher id (keep lower id).

INTUITION: DELETE with self-join: identify duplicate pairs, condition decides which to delete (higher id) vs keep (lower id).

PATTERN TIP: Always test DELETE with SELECT first. DELETE alias FROM table alias JOIN ... is MySQL multi-table DELETE syntax.

Q16. Trips and Users - Cancellation Rate

Hard
LeetCode, JP Morgan

PROBLEM STATEMENT

Compute cancellation rate of trips by unbanned users, Oct 1-3 2013. Round to 2 decimal places.

Table: Trips

Column Name	Type	Description
id	INT	Trip ID
client_id	INT	Client user ID
driver_id	INT	Driver user ID
status	VARCHAR	completed / cancelled_by_driver / cancelled_by_client
request_at	DATE	Date

Table: Users

Column Name	Type	Description
users_id	INT	User ID
banned	VARCHAR	Yes or No
role	VARCHAR	client / driver

SOLUTION 1: JOIN unbanned + SUM CASE

```
SELECT t.request_at AS Day,
       ROUND(
         SUM(CASE WHEN t.status != 'completed' THEN 1 ELSE 0 END) / COUNT(*),
       2) AS 'Cancellation Rate'
FROM Trips t
JOIN Users u_c ON t.client_id  = u_c.users_id AND u_c.banned = 'No'
JOIN Users u_d ON t.driver_id  = u_d.users_id AND u_d.banned = 'No'
WHERE t.request_at BETWEEN '2013-10-01' AND '2013-10-03'
GROUP BY t.request_at
ORDER BY t.request_at;
```

Explanation:

- JOIN Users twice: filter unbanned client AND driver in JOIN condition.
- CASE: 1 for cancelled, 0 for completed.
- SUM / COUNT = cancellation rate.
- ROUND(x, 2): 2 decimal places.

INTUITION: Multi-join same table with different aliases for client and driver. Conditional rate = SUM(CASE) / COUNT(*).

PATTERN TIP: Rate = SUM(condition as 0/1) / COUNT(*). ROUND(x, N) rounds to N decimal places.

Q17. Game Play Analysis - First Login

Easy
LeetCode

PROBLEM STATEMENT

Report the first login date for each player.

Table: Activity

Column Name	Type	Description
player_id	INT	Player ID
device_id	INT	Device
event_date	DATE	Date
games_played	INT	Games played

SOLUTION 1: MIN() GROUP BY

```
SELECT player_id, MIN(event_date) AS first_login
FROM Activity
GROUP BY player_id;
```

Explanation:

- MIN(event_date): earliest date per player.
- GROUP BY player_id: one row per player.

INTUITION: First occurrence = MIN(date) GROUP BY entity. The most fundamental aggregation pattern.

PATTERN TIP: MIN/MAX on DATE: MIN = earliest, MAX = latest.

Q18. Managers with 5+ Direct Reports

Medium
JP Morgan, LeetCode

PROBLEM STATEMENT

Find managers who have at least 5 direct reports.

Table: Employee

Column Name	Type	Description
id	INT	Employee ID
name	VARCHAR	Name
managerId	INT	Manager ID (nullable)

SOLUTION 1: GROUP BY + HAVING + JOIN

```
SELECT e.name
FROM Employee e
JOIN (
  SELECT managerId
  FROM Employee
  WHERE managerId IS NOT NULL
  GROUP BY managerId
  HAVING COUNT(*) >= 5
) mgr ON e.id = mgr.managerId;
```

Explanation:

- Inner query: managers with 5+ reports using HAVING.
- IS NOT NULL: exclude null manager references.
- Outer JOIN: get manager name.

INTUITION: Two steps: aggregate to find threshold-meeting IDs, then join to get their names.

PATTERN TIP: HAVING COUNT(*) >= 5: at least 5 rows in the group.

Q19. Exchange Seats - Swap Adjacent Students

Medium
LeetCode

PROBLEM STATEMENT

Swap seat ids of every two consecutive students. If last student has odd id, keep it unchanged.

Table: Seat

Column Name	Type	Description

id	INT	Seat ID (sequential)
student	VARCHAR	Student name

SOLUTION 1: CASE WHEN MOD

```
SELECT
  CASE
    WHEN id % 2 = 1 AND id = (SELECT COUNT(*) FROM Seat) THEN id
    WHEN id % 2 = 1 THEN id + 1
    ELSE id - 1
  END AS id,
  student
FROM Seat
ORDER BY id;
```

Explanation:

- id % 2 = 1: odd seat.
- Last odd row: equals total count - keep unchanged.
- Odd (not last): move to id+1.
- Even: move to id-1.

INTUITION: Odd to id+1, even to id-1. Edge case: last odd seat stays. CASE for three scenarios.

PATTERN TIP: MOD(id,2) = id%2: 0=even, 1=odd. Subquery for total count handles the edge case.

Q20. Students and Examinations - Zero Counts

Medium
LeetCode, UBS

PROBLEM STATEMENT

Find how many times each student attended each exam. Include 0 for combinations not attended.

Table: Students

Column Name	Type	Description
student_id	INT	Student ID
student_name	VARCHAR	Name

Table: Subjects

Column Name	Type	Description
subject_name	VARCHAR	Subject name

Table: Examinations

Column Name	Type	Description
student_id	INT	Student ID
subject_name	VARCHAR	Subject taken

SOLUTION 1: CROSS JOIN + LEFT JOIN + COUNT

```
SELECT s.student_id, s.student_name, sub.subject_name,
       COUNT(e.student_id) AS attended_exams
FROM Students s
CROSS JOIN Subjects sub
LEFT JOIN Examinations e
  ON s.student_id = e.student_id
  AND sub.subject_name = e.subject_name
GROUP BY s.student_id, s.student_name, sub.subject_name
ORDER BY s.student_id, sub.subject_name;
```

Explanation:

- CROSS JOIN: all student x subject combinations.
- LEFT JOIN: actual attendance, NULLs for missing.
- COUNT(e.student_id): counts non-NULL (real exams). Returns 0 for absent.

INTUITION: All combinations with zero-counts: CROSS JOIN to generate all pairs, LEFT JOIN fact table, COUNT non-NULLs.

PATTERN TIP: CROSS JOIN = Cartesian product. COUNT(col) ignores NULLs - perfect for zero-aware counting after LEFT JOIN.

SECTION 3

Aggregation, GROUP BY and Date Functions

Q21. Count Salary Categories

Medium
LeetCode, JP Morgan

PROBLEM STATEMENT

Count bank accounts per category: Low Salary (<20000), Average Salary (20000-50000), High Salary (>50000). Include all 3 even if count is 0.

Table: Accounts

Column Name	Type	Description
account_id	INT	Account ID
income	INT	Monthly income

SOLUTION 1: UNION ALL (row-per-category)

```
SELECT 'Low Salary' AS category, COUNT(*) AS accounts_count
FROM Accounts WHERE income < 20000
UNION ALL
SELECT 'Average Salary', COUNT(*) FROM Accounts WHERE income BETWEEN 20000 AND
50000
UNION ALL
SELECT 'High Salary', COUNT(*) FROM Accounts WHERE income > 50000;
```

Explanation:

• Three separate COUNT queries per bucket.
• UNION ALL: combine into one result.
• COUNT(*) returns 0 for empty buckets - covers 'include all' requirement.

SOLUTION 2: SUM CASE (single scan)

```
SELECT
  SUM(CASE WHEN income < 20000 THEN 1 ELSE 0 END) AS low_salary,
  SUM(CASE WHEN income BETWEEN 20000 AND 50000 THEN 1 ELSE 0 END) AS avg_salary,
  SUM(CASE WHEN income > 50000 THEN 1 ELSE 0 END) AS high_salary
FROM Accounts;
```

Explanation:
• One scan, three conditional sums.
• Output is 3 columns (wide) vs UNION which gives 3 rows.

INTUITION: Fixed category counting: UNION ALL for row-format, SUM(CASE) for column-format. SUM(CASE WHEN cond THEN 1 ELSE 0 END) = conditional count.

PATTERN TIP: SUM(CASE WHEN condition THEN 1 ELSE 0 END) = COUNT(*) WHERE condition. One of the most versatile SQL patterns.

Q22. Capital Gain/Loss per Stock

Medium
LeetCode, UBS

PROBLEM STATEMENT

Report capital gain/loss = total sell price - total buy price for each stock.

Table: Stocks

Column Name	Type	Description
stock_name	VARCHAR	Stock name
operation	VARCHAR	Buy or Sell
operation_day	INT	Day number
price	INT	Price

SOLUTION 1: SUM CASE signed amounts

```
SELECT stock_name,
       SUM(CASE WHEN operation = 'Sell' THEN price ELSE -price END) AS
capital_gain_loss
FROM Stocks
GROUP BY stock_name;
```

Explanation:
• Sell: +price (income). Buy: -price (cost).
• SUM of signed values = net gain or loss per stock.

INTUITION: Net gain = sells - buys. Sign amounts with CASE WHEN, SUM everything. Single-pass solution.

PATTERN TIP: SUM(CASE WHEN sell THEN +price ELSE -price END) = SUM(sells) - SUM(buys) in one pass.

Q23. Monthly Transactions Summary

Medium
LeetCode, Barclays

PROBLEM STATEMENT

For each month and country: total count, approved count, total amount, approved amount.

Table: Transactions

Column Name	Type	Description
id	INT	Transaction ID
country	VARCHAR	Country
state	VARCHAR	approved or declined
amount	INT	Amount
trans_date	DATE	Date

SOLUTION 1: DATE_FORMAT + SUM CASE

```
SELECT DATE_FORMAT(trans_date,'%Y-%m') AS month, country,
       COUNT(*) AS trans_count,
       SUM(CASE WHEN state='approved' THEN 1 ELSE 0 END) AS approved_count,
       SUM(amount) AS trans_total_amount,
       SUM(CASE WHEN state='approved' THEN amount ELSE 0 END) AS
approved_total_amount
FROM Transactions
GROUP BY DATE_FORMAT(trans_date,'%Y-%m'), country;
```

Explanation:

- DATE_FORMAT '%Y-%m': group by year-month.
- SUM(CASE WHEN state='approved'): conditional totals.

INTUITION: Conditional aggregation with SUM(CASE WHEN) is the SQL pivot technique. Multiple metrics in one GROUP BY.

PATTERN TIP: SUM(CASE WHEN state='approved' THEN value ELSE 0 END) = filtered aggregation. SQL's built-in conditional summing.

Q24. Group Sold Products by Date (GROUP_CONCAT)

Easy
LeetCode, Barclays

PROBLEM STATEMENT

For each date: distinct product count and alphabetically sorted comma-separated product names.

Table: Activities

Column Name	Type	Description
sell_date	DATE	Date of sale
product	VARCHAR	Product name

SOLUTION 1: GROUP_CONCAT + COUNT DISTINCT

```
SELECT sell_date,
       COUNT(DISTINCT product) AS num_sold,
       GROUP_CONCAT(DISTINCT product ORDER BY product SEPARATOR ',') AS products
FROM Activities
GROUP BY sell_date
ORDER BY sell_date;
```

Explanation:

• COUNT(DISTINCT product): unique products per date.
• GROUP_CONCAT(DISTINCT ... ORDER BY ... SEPARATOR ','): sorted unique comma-separated list.
• MySQL-specific - very common in assessments.

INTUITION: GROUP_CONCAT is MySQL's string aggregation function. DISTINCT removes duplicates, ORDER BY sorts within the concat.

PATTERN TIP: GROUP_CONCAT syntax: GROUP_CONCAT([DISTINCT] col [ORDER BY col] [SEPARATOR 'sep']). Default separator is comma.

Q25. Customers with Positive Revenue 2021

Easy
LeetCode

PROBLEM STATEMENT

Find customers with positive total revenue in the year 2021.

Table: Customers

Column Name	Type	Description
customer_id	INT	Customer ID
year	INT	Year
revenue	INT	Revenue (can be negative)

SOLUTION 1: WHERE + GROUP BY + HAVING

```
SELECT customer_id
FROM Customers
WHERE year = 2021
GROUP BY customer_id
HAVING SUM(revenue) > 0;
```

Explanation:

• WHERE year=2021: filter to 2021 rows.
• GROUP BY: per customer.
• HAVING SUM > 0: positive total.

INTUITION: Filter (WHERE) -> aggregate (GROUP BY) -> filter aggregate (HAVING). This pipeline is foundational SQL.

PATTERN TIP: SQL execution order: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT.

Q26. Employees Hired in Last 30 Days

Easy
General

PROBLEM STATEMENT

Find all employees hired in the last 30 days.

Table: Employees

Column Name	Type	Description
emp_id	INT	Employee ID
name	VARCHAR	Name
hire_date	DATE	Hire date

SOLUTION 1: DATE_SUB (index-friendly)

```
SELECT * FROM Employees
WHERE hire_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY);
```

Explanation:

- DATE_SUB(CURDATE(), INTERVAL 30 DAY): date 30 days before today.
- hire_date >=: hired on or after that date.
- More index-friendly than DATEDIFF approach.

SOLUTION 2: DATEDIFF

```
SELECT * FROM Employees
WHERE DATEDIFF(CURDATE(), hire_date) <= 30;
```

Explanation:

- DATEDIFF(today, hire_date): days since hire.
- <= 30: within last 30 days.

INTUITION: Date range filtering: compute threshold date with DATE_SUB, compare directly. Prefer DATE_SUB approach for index usage.

PATTERN TIP: DATE_ADD/DATE_SUB: DATE_SUB(CURDATE(), INTERVAL N DAY/MONTH/YEAR). CURDATE()=today (no time). NOW()=current timestamp with time.

Q27. Calculate Age from Date of Birth

Easy
Barclays

PROBLEM STATEMENT

Calculate current age in completed years for each employee.

Table: Employees

Column Name	Type	Description
emp_id	INT	Employee ID
name	VARCHAR	Name
dob	DATE	Date of birth

SOLUTION 1: TIMESTAMPDIFF(YEAR,...)

```
SELECT emp_id, name,
       TIMESTAMPDIFF(YEAR, dob, CURDATE()) AS age
FROM Employees;
```

Explanation:

- TIMESTAMPDIFF(YEAR, earlier, later): complete years between two dates.
- Accounts for leap years and whether birthday has passed this year.

INTUITION: Age = TIMESTAMPDIFF(YEAR, dob, today). More accurate than DATEDIFF/365 - respects calendar boundaries.

PATTERN TIP: TIMESTAMPDIFF(unit, date1, date2): unit can be YEAR, MONTH, DAY, HOUR, MINUTE, SECOND.

Q28. Find Weekend Transactions

Easy
Barclays

PROBLEM STATEMENT

Find all transactions that occurred on Saturday or Sunday.

Table: Transactions

Column Name	Type	Description
txn_id	INT	Transaction ID
txn_date	DATE	Date
amount	DECIMAL	Amount

SOLUTION 1: DAYOFWEEK() IN (1,7)

```
SELECT * FROM Transactions
WHERE DAYOFWEEK(txn_date) IN (1, 7);
-- DAYOFWEEK: 1=Sunday, 7=Saturday
```

Explanation:

- DAYOFWEEK(): 1=Sunday, 2=Monday ... 7=Saturday.
- IN (1,7): Sunday or Saturday = weekend.

INTUITION: Day-of-week filtering: DAYOFWEEK returns 1-7. 1=Sunday, 7=Saturday.

PATTERN TIP: DAYOFWEEK: 1=Sun, 7=Sat. WEEKDAY: 0=Mon, 6=Sun. DAYNAME(): returns 'Saturday'. Different reference days.

Q29. Year-over-Year Revenue Growth

Medium
JP Morgan, Barclays

PROBLEM STATEMENT

Calculate YoY revenue growth percentage for each year.

Table: Orders

Column Name	Type	Description
order_id	INT	Order ID
order_date	DATE	Date
revenue	DECIMAL	Revenue

SOLUTION 1: LAG Window Function

```
SELECT year, revenue,
       LAG(revenue) OVER (ORDER BY year) AS prev_year_revenue,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY year))
             / LAG(revenue) OVER (ORDER BY year), 2) AS yoy_growth_pct
FROM (
  SELECT YEAR(order_date) AS year, SUM(revenue) AS revenue
  FROM Orders
  GROUP BY YEAR(order_date)
) yearly
ORDER BY year;
```

Explanation:

- Inner: aggregate revenue by year.
- LAG(revenue): previous year revenue.
- (current - prior) / prior * 100: growth %.
- First year: LAG returns NULL.

INTUITION: YoY = aggregate by year, then LAG to access prior year. Growth = (current - prior) / prior x 100.

PATTERN TIP: LAG(col, 1): one row back. LEAD: one row forward. First row LAG = NULL.

Q30. Quarterly Revenue Breakdown

Medium
JP Morgan, Barclays

PROBLEM STATEMENT

Find total revenue for each quarter of each year.

Table: Sales

Column Name	Type	Description
sale_id	INT	Sale ID
sale_date	DATE	Sale date
revenue	DECIMAL	Revenue

SOLUTION 1: YEAR + QUARTER

```
SELECT YEAR(sale_date)    AS year,
       QUARTER(sale_date) AS quarter,
       SUM(revenue)       AS total_revenue
FROM Sales
GROUP BY YEAR(sale_date), QUARTER(sale_date)
ORDER BY year, quarter;
```

Explanation:

• QUARTER(): returns 1-4. Q1=Jan-Mar, Q2=Apr-Jun, Q3=Jul-Sep, Q4=Oct-Dec.
• GROUP BY both YEAR and QUARTER: Q1-2023 != Q1-2024.

INTUITION: QUARTER() maps months to 1-4. Always GROUP BY both YEAR and QUARTER for multi-year data.

PATTERN TIP: MySQL date extraction: YEAR(), MONTH(), DAY(), QUARTER(), WEEK(). All return integers.

SECTION 4

Window Functions and Subqueries

Q31. Top 3 Highest-Paid per Department

Medium
JP Morgan, UBS

PROBLEM STATEMENT

Find the top 3 highest-paid employees in each department.

Table: Employee

Column Name	Type	Description
emp_id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary
dept_id	INT	Department ID

SOLUTION 1: ROW_NUMBER PARTITION BY

```
SELECT dept_id, name, salary
```

```
FROM (
  SELECT dept_id, name, salary,
         ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
  FROM Employee
) t
WHERE rn <= 3
ORDER BY dept_id, salary DESC;
```

Explanation:
• PARTITION BY dept_id: row number resets per department.
• ORDER BY salary DESC: rank 1 = highest.
• WHERE rn <= 3: top 3 per department.

INTUITION: Top-N per group: ROW_NUMBER PARTITION BY group ORDER BY metric DESC, then WHERE rn <= N.

PATTERN TIP: ROW_NUMBER for strict top-N. DENSE_RANK if ties should be included. Choose based on requirement.

Q32. 3-Day Moving Average

Medium
JP Morgan, Barclays

PROBLEM STATEMENT
Calculate the 3-day moving average of daily sales.

Table: DailySales

Column Name	Type	Description
sale_date	DATE	Date
amount	DECIMAL	Daily sales amount

SOLUTION 1: AVG OVER ROWS BETWEEN
```
SELECT sale_date, amount,
       ROUND(AVG(amount) OVER (
         ORDER BY sale_date
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ), 2) AS moving_avg_3day
FROM DailySales
ORDER BY sale_date;
```

Explanation:
• ROWS BETWEEN 2 PRECEDING AND CURRENT ROW: window = current + 2 prior rows = 3 rows total.
• AVG over this window = 3-day moving average.

INTUITION: Moving average: AVG with sliding window. ROWS BETWEEN N PRECEDING AND CURRENT ROW = N+1 row window.

PATTERN TIP: Window frame: ROWS BETWEEN start AND end. Options: UNBOUNDED PRECEDING, N PRECEDING, CURRENT ROW, N FOLLOWING, UNBOUNDED FOLLOWING.

Q33. Running Total (Cumulative Sum)

Medium
JP Morgan

PROBLEM STATEMENT
Show cumulative total of sales ordered by date.

Table: DailySales

Column Name	Type	Description
sale_date	DATE	Date
amount	DECIMAL	Daily sales

SOLUTION 1: SUM OVER ORDER BY

```
SELECT sale_date, amount,
       SUM(amount) OVER (ORDER BY sale_date) AS running_total
FROM DailySales
ORDER BY sale_date;
```

Explanation:
• SUM OVER (ORDER BY): cumulative sum from first row to current.
• No PARTITION BY: single running total across all dates.

INTUITION: Running total = SUM OVER (ORDER BY time). Accumulates from start to current row.

PATTERN TIP: SUM OVER (ORDER BY): running total. Add PARTITION BY g: running total resets per group g.

Q34. Difference from Department Average

Medium
UBS, Barclays

PROBLEM STATEMENT
For each employee, show salary and how much it differs from their department average.

Table: Employee

Column Name	Type	Description
emp_id	INT	Employee ID
name	VARCHAR	Name
salary	INT	Salary
dept_id	INT	Department ID

SOLUTION 1: AVG OVER PARTITION BY

```
SELECT dept_id, name, salary,
```

```
       ROUND(AVG(salary) OVER (PARTITION BY dept_id), 2) AS dept_avg,
       salary - ROUND(AVG(salary) OVER (PARTITION BY dept_id), 2) AS
diff_from_avg
FROM Employee
ORDER BY dept_id, salary DESC;
```

Explanation:
• AVG OVER (PARTITION BY dept_id): dept average attached to every row.
• salary - dept_avg: deviation from average.
• No GROUP BY needed - all rows preserved.

INTUITION: Row value vs group aggregate: only window functions can attach group aggregate to individual rows. GROUP BY would collapse rows.

PATTERN TIP: Window functions keep all rows. GROUP BY collapses rows. For row-vs-group comparison: always use window functions.

Q35. Consecutive Login Streak (Islands Technique)

Hard
JP Morgan, UBS

PROBLEM STATEMENT
Find the longest consecutive daily login streak for each user.

Table: Logins

Column Name	Type	Description
user_id	INT	User ID
login_date	DATE	Login date

SOLUTION 1: date minus ROW_NUMBER island grouping

```
SELECT user_id, MAX(streak) AS longest_streak
FROM (
  SELECT user_id, COUNT(*) AS streak
  FROM (
    SELECT user_id, login_date,
           DATE_SUB(login_date,
             INTERVAL ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY
login_date) DAY
           ) AS grp
    FROM (SELECT DISTINCT user_id, login_date FROM Logins) t
  ) t2
  GROUP BY user_id, grp
) t3
GROUP BY user_id;
```

Explanation:
• DISTINCT: remove duplicate login dates.
• ROW_NUMBER per user ordered by date: sequential rank 1,2,3...
• date - ROW_NUMBER days: consecutive dates produce the SAME grp constant.
• GROUP BY grp: each consecutive run = one island group.
• COUNT(*): streak length. MAX: longest streak.

INTUITION: Islands technique: date - ROW_NUMBER = constant for consecutive dates. Groups consecutive rows together. Count per island = streak length.

PATTERN TIP: Islands trick: for consecutive integers/dates, value - ROW_NUMBER = constant within each consecutive group. A widely-used advanced pattern.

Q36. Products Never Ordered

Easy
LeetCode

PROBLEM STATEMENT

Find product names that have never been ordered.

Table: Products

Column Name	Type	Description
product_id	INT	Product ID
product_name	VARCHAR	Name

Table: Orders

Column Name	Type	Description
order_id	INT	Order ID
product_id	INT	FK to Products

SOLUTION 1: LEFT JOIN IS NULL

```
SELECT p.product_name
FROM Products p
LEFT JOIN Orders o ON p.product_id = o.product_id
WHERE o.product_id IS NULL;
```

Explanation:

- LEFT JOIN: all products, NULLs where no order found.
- IS NULL: products with no matching order.

INTUITION: No match in another table = LEFT JOIN IS NULL (anti-join pattern).

PATTERN TIP: LEFT JOIN IS NULL = NOT EXISTS = NOT IN. LEFT JOIN IS NULL is most efficient and handles NULLs correctly.

Q37. Products Sold in Every Store

Medium
LeetCode

PROBLEM STATEMENT

Find products that are sold in every store.

Table: Products

Column Name	Type	Description
product_id	INT	Product ID
store	VARCHAR	Store name

SOLUTION 1: COUNT DISTINCT = total stores

```
SELECT product_id
FROM Products
GROUP BY product_id
HAVING COUNT(DISTINCT store) = (
  SELECT COUNT(DISTINCT store) FROM Products
);
```

Explanation:

- COUNT(DISTINCT store): unique stores carrying this product.
- Subquery: total unique stores.
- HAVING equal: product is in ALL stores.

INTUITION: Appears in all groups: COUNT(DISTINCT group_col) = total groups. SQL's way of expressing universal quantification.

PATTERN TIP: No SQL FOR ALL operator. Workaround: GROUP BY entity, CHECK COUNT(DISTINCT group) = total groups.

Q38. NOT EXISTS - Employees with No Projects

Medium
General

PROBLEM STATEMENT

Find employees who have never been assigned to any project.

Table: Employees

Column Name	Type	Description
emp_id	INT	Employee ID
name	VARCHAR	Name

Table: ProjectAssignments

Column Name	Type	Description
project_id	INT	Project ID
emp_id	INT	Employee ID

SOLUTION 1: NOT EXISTS

```
SELECT e.emp_id, e.name
FROM Employees e
WHERE NOT EXISTS (
  SELECT 1 FROM ProjectAssignments pa
  WHERE pa.emp_id = e.emp_id
);
```

Explanation:
• NOT EXISTS: true if subquery returns zero rows.
• SELECT 1: we only need to know if any row exists.
• Correlated: runs once per employee row.

INTUITION: NOT EXISTS is the most readable anti-join. Stops at first match (efficient). Correctly handles NULLs.

PATTERN TIP: EXISTS/NOT EXISTS stops scanning after first match. More efficient than COUNT > 0. Handles NULLs correctly unlike NOT IN.

Q39. CASE WHEN Transaction Categories

Easy
Barclays

PROBLEM STATEMENT

Categorise each transaction: amount < 1000 = Small, 1000-10000 = Medium, > 10000 = Large.

Table: Transactions

Column Name	Type	Description
txn_id	INT	Transaction ID
amount	DECIMAL	Amount

SOLUTION 1: CASE WHEN THEN ELSE END

```
SELECT txn_id, amount,
       CASE
         WHEN amount < 1000 THEN 'Small'
         WHEN amount BETWEEN 1000 AND 10000 THEN 'Medium'
         ELSE 'Large'
       END AS category
FROM Transactions;
```

Explanation:
• CASE WHEN: SQL if-then-else. Evaluates top to bottom.
• First match wins.
• ELSE: default for unmatched rows.

INTUITION: CASE WHEN is SQL conditional logic. Conditions evaluated top-to-bottom. ELSE handles unmatched rows.

PATTERN TIP: CASE WHEN can appear in SELECT, WHERE, ORDER BY, GROUP BY, HAVING. Universal SQL conditional expression.

Q40. Dormant Accounts (90+ Days Inactive)

Medium
Barclays

PROBLEM STATEMENT

Find accounts with no transactions in the past 90 days.

Table: Transactions

Column Name	Type	Description
txn_id	INT	Transaction ID
account_id	INT	Account ID
txn_date	DATE	Date

SOLUTION 1: MAX date + DATEDIFF HAVING

```
SELECT account_id,
       MAX(txn_date) AS last_txn_date,
       DATEDIFF(CURDATE(), MAX(txn_date)) AS days_inactive
FROM Transactions
GROUP BY account_id
HAVING DATEDIFF(CURDATE(), MAX(txn_date)) >= 90
ORDER BY days_inactive DESC;
```

Explanation:

- MAX(txn_date): most recent transaction per account.
- DATEDIFF: days since last transaction.
- HAVING >= 90: dormant threshold filter.

INTUITION: Dormant: MAX(last_activity) per account, DATEDIFF for inactivity, HAVING for threshold.

PATTERN TIP: HAVING can use complex expressions like DATEDIFF(CURDATE(), MAX(col)) directly. No subquery needed.

SECTION 5

Company Assessment Patterns (Medium–Advanced)

Q41. Portfolio Risk Concentration

Advanced
Barclays, UBS

PROBLEM STATEMENT

Find clients with more than 50% of portfolio in a single sector. Return client_id, sector, sector_value, total_value, concentration_pct.

Table: Holdings

Column Name	Type	Description

client_id	INT	Client ID
sector	VARCHAR	Sector name
holding_value	DECIMAL	Value in this sector

SOLUTION 1: Nested window SUM + ratio filter

```
SELECT client_id, sector, sector_value, total_value,
       ROUND(100 * sector_value / total_value, 2) AS concentration_pct
FROM (
  SELECT client_id, sector,
         SUM(holding_value) AS sector_value,
         SUM(SUM(holding_value)) OVER (PARTITION BY client_id) AS total_value
  FROM Holdings
  GROUP BY client_id, sector
) t
WHERE sector_value / total_value > 0.5
ORDER BY concentration_pct DESC;
```

Explanation:

• SUM(holding_value): sector total per client-sector.
• SUM(SUM(holding_value)) OVER (PARTITION BY client_id): nested window on GROUP BY result = total portfolio per client.
• WHERE ratio > 0.5: concentrated sectors.

INTUITION: Risk concentration: aggregate by group, apply OVER on top of GROUP BY aggregate for total, compute ratio.

PATTERN TIP: SUM(SUM(col)) OVER (PARTITION BY g): outer SUM sees grouped rows, totals across them per partition. Advanced pattern tested at Barclays/UBS.

Q42. Trade Settlement Efficiency (T+2)

Advanced
UBS, Barclays

PROBLEM STATEMENT

Per trader: total trades, avg settlement days, % settled within T+2. Flag traders below 80% T+2.

Table: Trades

Column Name	Type	Description
trade_id	INT	Trade ID
trader_id	INT	Trader ID
trade_date	DATE	Trade date
settlement_date	DATE	Settlement date

SOLUTION 1: DATEDIFF + conditional rate + CASE flag

```
SELECT trader_id,
       COUNT(*) AS total_trades,
       ROUND(AVG(DATEDIFF(settlement_date, trade_date)), 2) AS
avg_settlement_days,
       ROUND(100 * SUM(CASE WHEN DATEDIFF(settlement_date, trade_date) <= 2
```

```
                              THEN 1 ELSE 0 END) / COUNT(*), 2) AS pct_t_plus_2,
       CASE WHEN SUM(CASE WHEN DATEDIFF(settlement_date, trade_date) <= 2
                     THEN 1 ELSE 0 END) / COUNT(*) < 0.8
            THEN 'FLAG' ELSE 'OK' END AS status
FROM Trades
GROUP BY trader_id
ORDER BY pct_t_plus_2;
```

Explanation:

- DATEDIFF: days to settle each trade.
- AVG: average settlement days per trader.
- SUM(CASE WHEN <= 2) / COUNT * 100: T+2 rate.
- CASE: flag traders below 80%.

INTUITION: Settlement efficiency: per-row DATEDIFF, aggregate with AVG and conditional SUM for rate, CASE for flagging.

PATTERN TIP: T+N settlement: DATEDIFF(settlement_date, trade_date) <= N. Conditional rate = SUM(CASE)/COUNT(*).

Q43. Loan Portfolio Aging Buckets

Advanced
Barclays

PROBLEM STATEMENT

Classify overdue loans: 1-30 days, 31-60 days, 61-90 days, 90+ days overdue. Count and sum per bucket.

Table: Loans

Column Name	Type	Description
loan_id	INT	Loan ID
customer_id	INT	Customer ID
due_date	DATE	Payment due date
outstanding_balance	DECIMAL	Balance outstanding
status	VARCHAR	overdue or paid

SOLUTION 1: DATEDIFF + CASE bucket + GROUP BY

```
SELECT
  CASE
    WHEN DATEDIFF(CURDATE(), due_date) BETWEEN 1  AND 30 THEN '1-30 days'
    WHEN DATEDIFF(CURDATE(), due_date) BETWEEN 31 AND 60 THEN '31-60 days'
    WHEN DATEDIFF(CURDATE(), due_date) BETWEEN 61 AND 90 THEN '61-90 days'
    ELSE '90+ days'
  END AS aging_bucket,
  COUNT(*) AS loan_count,
  SUM(outstanding_balance) AS total_outstanding
FROM Loans
WHERE status = 'overdue'
GROUP BY aging_bucket
ORDER BY MIN(DATEDIFF(CURDATE(), due_date));
```

Explanation:

• WHERE status='overdue': only overdue loans.
• DATEDIFF: days past due.
• CASE: assigns bucket label.
• ORDER BY MIN(days): chronological bucket order (not alphabetical).

INTUITION: Loan aging = core credit risk SQL. DATEDIFF for days overdue, CASE to bucket, GROUP BY bucket.

PATTERN TIP: ORDER BY string bucket label gives alphabetical order. ORDER BY MIN(numeric metric) gives chronological order. Always order aging buckets numerically.

Q44. Customer Churn Analysis

Advanced
Barclays

PROBLEM STATEMENT

Identify churned customers: active in Apr-Jun 2023 but no orders in Jul-Sep 2023.

Table: Orders

Column Name	Type	Description
order_id	INT	Order ID
customer_id	INT	Customer ID
order_date	DATE	Order date

SOLUTION 1: EXISTS + NOT EXISTS

```
SELECT DISTINCT customer_id
FROM Orders o1
WHERE EXISTS (
  SELECT 1 FROM Orders o2
  WHERE o2.customer_id = o1.customer_id
    AND o2.order_date BETWEEN '2023-04-01' AND '2023-06-30'
)
AND NOT EXISTS (
  SELECT 1 FROM Orders o3
  WHERE o3.customer_id = o1.customer_id
    AND o3.order_date BETWEEN '2023-07-01' AND '2023-09-30'
);
```

Explanation:
• EXISTS: at least one order in pre-cutoff period.
• NOT EXISTS: zero orders in post-cutoff period.
• Combined with AND: churned definition.

INTUITION: Churn = active before AND inactive after. EXISTS for 'at least one', NOT EXISTS for 'none'.

PATTERN TIP: EXISTS stops at first match. NOT EXISTS = zero matching rows. Combine with AND for 'was active, now inactive'.

Q45. Counterparty Exposure Netting

Advanced
UBS, Barclays

PROBLEM STATEMENT

Find net exposure per counterparty (BUY minus SELL notional). Flag those with net > 1,000,000.

Table: Trades

Column Name	Type	Description
trade_id	INT	Trade ID
counterparty	VARCHAR	Counterparty name
direction	VARCHAR	BUY or SELL
notional	DECIMAL	Trade notional amount

SOLUTION 1: SUM CASE netting + CASE flag

```
SELECT counterparty,
       SUM(CASE WHEN direction='BUY' THEN notional ELSE -notional END) AS
net_exposure,
       CASE WHEN SUM(CASE WHEN direction='BUY' THEN notional ELSE -notional END)
> 1000000
            THEN 'FLAGGED' ELSE 'OK' END AS status
FROM Trades
GROUP BY counterparty
ORDER BY net_exposure DESC;
```

Explanation:

- BUY: +notional. SELL: -notional.
- SUM of signed notionals = net exposure.
- CASE flag: net > 1M is flagged.

INTUITION: Financial netting = SUM of signed amounts. BUY positive, SELL negative. Used in market risk, credit exposure, P&L.

PATTERN TIP: Netting: SUM(CASE WHEN direction='BUY' THEN amount ELSE -amount END). Positive = long, negative = short.

Q46. Intraday Position Limit Breach

Advanced
UBS

PROBLEM STATEMENT

Find traders who exceeded a net position limit of 10,000 units at any point during the day.

Table: Trades

Column Name	Type	Description
trade_id	INT	Trade ID

trader_id	INT	Trader ID
trade_time	DATETIME	Trade timestamp
direction	VARCHAR	BUY or SELL
quantity	INT	Quantity

SOLUTION 1: Running signed SUM window

```
SELECT DISTINCT trader_id
FROM (
  SELECT trader_id, trade_time,
         SUM(CASE WHEN direction='BUY' THEN quantity ELSE -quantity END)
           OVER (PARTITION BY trader_id, DATE(trade_time)
                 ORDER BY trade_time
                 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
running_position
  FROM Trades
) t
WHERE running_position > 10000;
```

Explanation:

- CASE signed quantity: BUY +, SELL -.
- SUM OVER PARTITION BY trader + date: per-trader per-day running position.
- ORDER BY trade_time: chronological sequence.
- WHERE > 10000: at any point exceeded limit.

INTUITION: Intraday position = cumulative signed sum ordered by time, partitioned by trader+day. Any row exceeding limit = breach.

PATTERN TIP: PARTITION BY trader AND date: resets running total each day. ORDER BY time: correct intraday sequence.

Q47. VWAP - Volume Weighted Average Price

Advanced
UBS, Barclays

PROBLEM STATEMENT

Calculate VWAP per security per day. VWAP = SUM(price x volume) / SUM(volume).

Table: Trades

Column Name	Type	Description
trade_id	INT	Trade ID
security_id	VARCHAR	Security symbol
trade_date	DATE	Trade date
price	DECIMAL	Trade price
volume	INT	Volume traded

SOLUTION 1: SUM(price * vol) / SUM(vol)

```
SELECT security_id, trade_date,
       ROUND(SUM(price * volume) / SUM(volume), 4) AS vwap,
       SUM(volume) AS total_volume
```

```
FROM Trades
GROUP BY security_id, trade_date
ORDER BY security_id, trade_date;
```

Explanation:
• SUM(price * volume): total consideration (price x qty for each trade).
• SUM(volume): total volume.
• VWAP = total consideration / total volume.

INTUITION: VWAP = weighted average where weight = volume. SQL: SUM(value * weight) / SUM(weight). Standard trading analytics pattern.

PATTERN TIP: Weighted average: SUM(value * weight) / SUM(weight). Works for any weighting: volume, units, days, etc.

Q48. Point-in-Time FX Rate Application

Advanced
Barclays, UBS

PROBLEM STATEMENT

Convert transactions to USD using the most recent applicable exchange rate on or before the transaction date.

Table: Transactions

Column Name	Type	Description
txn_id	INT	Transaction ID
currency	VARCHAR	Currency code
amount	DECIMAL	Amount in local currency
txn_date	DATE	Transaction date

Table: ExchangeRates

Column Name	Type	Description
currency	VARCHAR	Currency code
rate_date	DATE	Rate effective date
usd_rate	DECIMAL	USD per 1 unit of currency

SOLUTION 1: JOIN on currency + correlated MAX date subquery

```
SELECT t.txn_id, t.currency, t.amount, t.txn_date,
       er.usd_rate,
       ROUND(t.amount * er.usd_rate, 2) AS usd_amount
FROM Transactions t
JOIN ExchangeRates er
  ON t.currency = er.currency
  AND er.rate_date = (
    SELECT MAX(rate_date) FROM ExchangeRates
    WHERE currency = t.currency
      AND rate_date <= t.txn_date
  );
```

Explanation:
• JOIN on currency: match transaction to rate table.
• Correlated subquery: MAX rate_date on or before txn_date = most recent applicable rate.
• amount * usd_rate: converted USD value.

INTUITION: Point-in-time rate: MAX(effective_date) WHERE effective_date <= target_date. Retrieves the most recently effective value as of the transaction date.

PATTERN TIP: Point-in-time lookup: SELECT value WHERE entity=X AND date<=target ORDER BY date DESC LIMIT 1. Essential for slowly-changing-dimension (SCD) tables.

Q49. KYC Document Expiry Alerts

Medium
Barclays

PROBLEM STATEMENT

Find customers with documents expired or expiring within 30 days. Categorise as Expired or Expiring Soon.

Table: KYCDocuments

Column Name	Type	Description
customer_id	INT	Customer ID
doc_type	VARCHAR	Document type
expiry_date	DATE	Document expiry date

SOLUTION 1: DATEDIFF + CASE categorisation

```
SELECT customer_id, doc_type, expiry_date,
       CASE
         WHEN expiry_date < CURDATE() THEN 'Expired'
         WHEN expiry_date <= DATE_ADD(CURDATE(), INTERVAL 30 DAY) THEN 'Expiring
Soon'
       END AS alert_status,
       DATEDIFF(expiry_date, CURDATE()) AS days_to_expiry
FROM KYCDocuments
WHERE expiry_date <= DATE_ADD(CURDATE(), INTERVAL 30 DAY)
ORDER BY expiry_date;
```

Explanation:
• WHERE: limit scope to relevant documents.
• CASE: categorise expired vs expiring soon.
• DATEDIFF(expiry, today): negative for expired, 0-30 for expiring soon.

INTUITION: Expiry alerts: compare to CURDATE(). DATEDIFF negative = expired. Filter with WHERE, categorise with CASE.

PATTERN TIP: DATE_ADD(CURDATE(), INTERVAL 30 DAY): 30 days from today. INTERVAL units: DAY, WEEK, MONTH, YEAR.

Q50. Loyalty Tier Upgrade Eligibility

Advanced
Barclays

PROBLEM STATEMENT

Silver-tier customers spending >= $5000 in last 6 months qualify for Gold upgrade. Return eligible customers with their total spend.

Table: Customers

Column Name	Type	Description
customer_id	INT	Customer ID
tier	VARCHAR	Silver / Gold / Platinum

Table: Transactions

Column Name	Type	Description
txn_id	INT	Transaction ID
customer_id	INT	Customer ID
txn_date	DATE	Transaction date
amount	DECIMAL	Amount spent

SOLUTION 1: JOIN + date filter + HAVING threshold

```
SELECT c.customer_id, c.tier,
       SUM(t.amount) AS spend_last_6m
FROM Customers c
JOIN Transactions t ON c.customer_id = t.customer_id
WHERE c.tier = 'Silver'
  AND t.txn_date >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
GROUP BY c.customer_id, c.tier
HAVING SUM(t.amount) >= 5000
ORDER BY spend_last_6m DESC;
```

Explanation:

- WHERE tier='Silver': dimension filter.
- DATE_SUB 6 months: last 6 months window.
- GROUP BY customer: aggregate spending.
- HAVING SUM >= 5000: upgrade threshold.

INTUITION: Multi-condition eligibility: dimension filter (WHERE), date range (WHERE), aggregate threshold (HAVING). Business rules map to SQL clauses.

PATTERN TIP: DATE_SUB(CURDATE(), INTERVAL 6 MONTH): 6 months ago. Combine WHERE (row filters) and HAVING (aggregate filters).

SECTION 6

Financial Domain Queries I

Q51. Rolling 7-Day Active Users

Advanced
Barclays, UBS

PROBLEM STATEMENT
For each date, count distinct users active in the 7-day window ending on that date.

Table: UserActivity

Column Name	Type	Description
user_id	INT	User ID
activity_date	DATE	Activity date

SOLUTION 1: Self-join 7-day DATEDIFF window

```
SELECT a.activity_date,
       COUNT(DISTINCT b.user_id) AS active_users_7d
FROM (SELECT DISTINCT activity_date FROM UserActivity) a
JOIN UserActivity b
  ON DATEDIFF(a.activity_date, b.activity_date) BETWEEN 0 AND 6
GROUP BY a.activity_date
ORDER BY a.activity_date;
```

Explanation:
- a: distinct anchor dates.
- DATEDIFF BETWEEN 0 AND 6: b is within 6 days before a (7-day window inclusive).
- COUNT DISTINCT user_id: unique users in window.

INTUITION: Rolling window = self-join with DATEDIFF range. Each anchor date collects N days of lookback activity.

PATTERN TIP: Rolling N-day window: DATEDIFF(anchor, other) BETWEEN 0 AND N-1. Include 0 for same-day activity.

Q52. Customer Cohort Retention

Advanced
JP Morgan

PROBLEM STATEMENT
For each signup cohort month, count total customers and how many made a purchase exactly 1 month after signup.

Table: Users

Column Name	Type	Description
user_id	INT	User ID
signup_date	DATE	Signup date

Table: Purchases

Column Name	Type	Description

user_id	INT	User ID
purchase_date	DATE	Purchase date

SOLUTION 1: LEFT JOIN + TIMESTAMPDIFF month = 1

```
SELECT DATE_FORMAT(u.signup_date,'%Y-%m') AS cohort_month,
       COUNT(DISTINCT u.user_id) AS cohort_size,
       COUNT(DISTINCT p.user_id) AS retained_month1
FROM Users u
LEFT JOIN Purchases p
  ON u.user_id = p.user_id
  AND TIMESTAMPDIFF(MONTH, u.signup_date, p.purchase_date) = 1
GROUP BY cohort_month
ORDER BY cohort_month;
```

Explanation:

- LEFT JOIN: includes all users even non-retained.
- TIMESTAMPDIFF(MONTH,...) = 1: exactly 1 month after signup.
- COUNT DISTINCT p.user_id: only those who returned (NULLs not counted).

INTUITION: Cohort retention: group by signup month, measure behaviour relative to signup using TIMESTAMPDIFF.

PATTERN TIP: TIMESTAMPDIFF(MONTH, signup, activity) = N gives month offset. Month 0 = signup month, Month 1 = first full month.

Q53. AML - Round Number Transaction Detection

Advanced
Barclays

PROBLEM STATEMENT

Find accounts where more than 50% of transactions are exact multiples of 1000 (AML structuring flag).

Table: Transactions

Column Name	Type	Description
txn_id	INT	Transaction ID
account_id	INT	Account ID
amount	DECIMAL	Transaction amount

SOLUTION 1: MOD + conditional rate + HAVING

```
SELECT account_id,
       COUNT(*) AS total_txns,
       SUM(CASE WHEN amount % 1000 = 0 THEN 1 ELSE 0 END) AS round_txns,
       ROUND(100.0 * SUM(CASE WHEN amount % 1000 = 0 THEN 1 ELSE 0 END) /
COUNT(*), 2) AS round_pct
FROM Transactions
GROUP BY account_id
HAVING 100.0 * SUM(CASE WHEN amount % 1000 = 0 THEN 1 ELSE 0 END) / COUNT(*) >
50
ORDER BY round_pct DESC;
```

Explanation:
• amount % 1000 = 0: divisible by 1000 = round number.
• SUM(CASE) / COUNT * 100: proportion of round transactions.
• HAVING: repeat expression (cannot use SELECT alias in HAVING).

INTUITION: AML detection: MOD for round number check, conditional rate for proportion. HAVING cannot reference SELECT aliases.

PATTERN TIP: HAVING cannot use SELECT aliases. Repeat the full expression or wrap in a subquery. MOD(n,d) = n%d = remainder.

Q54. Running Account Balance

Advanced
Barclays

PROBLEM STATEMENT
Show the running account balance after each transaction. Credits add, debits subtract.

Table: Transactions

Column Name	Type	Description
txn_id	INT	Transaction ID
account_id	INT	Account ID
txn_date	DATETIME	Transaction timestamp
txn_type	VARCHAR	debit or credit
amount	DECIMAL	Amount

SOLUTION 1: Signed SUM OVER PARTITION BY account

```
SELECT txn_id, account_id, txn_date, txn_type, amount,
       SUM(CASE WHEN txn_type='credit' THEN amount ELSE -amount END)
         OVER (PARTITION BY account_id ORDER BY txn_date, txn_id
               ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
running_balance
FROM Transactions
ORDER BY account_id, txn_date;
```

Explanation:
• CASE: credits positive, debits negative.
• SUM OVER PARTITION BY account_id: per-account running total.
• ORDER BY txn_date, txn_id: chronological with tiebreaker.

INTUITION: Running balance = cumulative signed SUM per account. PARTITION BY resets per account. ORDER BY time gives chronological accumulation.

PATTERN TIP: Running balance requires: (1) signed amounts via CASE, (2) cumulative SUM with ROWS frame, (3) PARTITION BY account.

Q55. Loan Default Risk Features

Advanced
Barclays

PROBLEM STATEMENT

Compute per customer: payment miss rate (% months paid < minimum) and average utilisation (balance / credit_limit).

Table: LoanPayments

Column Name	Type	Description
customer_id	INT	Customer ID
payment_month	DATE	Month (YYYY-MM-01)
payment_amount	DECIMAL	Amount paid
minimum_payment	DECIMAL	Minimum due
balance	DECIMAL	Outstanding balance
credit_limit	DECIMAL	Credit limit

SOLUTION 1: Multiple conditional aggregates in one GROUP BY

```
SELECT customer_id,
       ROUND(100 * SUM(CASE WHEN payment_amount < minimum_payment THEN 1 ELSE 0
END)
             / COUNT(*), 2) AS miss_rate_pct,
       ROUND(100 * AVG(balance / credit_limit), 2) AS avg_utilization_pct
FROM LoanPayments
GROUP BY customer_id
ORDER BY miss_rate_pct DESC;
```

Explanation:

- SUM(CASE WHEN payment < min) / COUNT * 100: miss rate percentage.
- AVG(balance / credit_limit) * 100: average utilisation rate.
- Both features computed in one GROUP BY pass.

INTUITION: SQL feature engineering: multiple risk metrics in one GROUP BY. Conditional rate + scalar ratio = two key credit risk features.

PATTERN TIP: Two feature patterns: (1) conditional rate = SUM(CASE)/COUNT, (2) ratio = AVG(col1/col2). Computable together in one GROUP BY.

Q56. Month-End Account Balance Report

Advanced
Barclays

PROBLEM STATEMENT

Find the balance at the end of each month per account (last transaction of each month).

Table: Transactions

Column Name	Type	Description
txn_id	INT	Transaction ID

account_id	INT	Account ID
txn_date	DATE	Transaction date
balance	DECIMAL	Balance after this transaction

SOLUTION 1: ROW_NUMBER DESC per account per month

```
SELECT account_id,
       DATE_FORMAT(txn_date,'%Y-%m') AS month,
       balance AS month_end_balance
FROM (
  SELECT account_id, txn_date, balance,
         ROW_NUMBER() OVER (
           PARTITION BY account_id, DATE_FORMAT(txn_date,'%Y-%m')
           ORDER BY txn_date DESC, txn_id DESC
         ) AS rn
  FROM Transactions
) t
WHERE rn = 1
ORDER BY account_id, month;
```

Explanation:

• PARTITION BY account + month: separate window per account per month.
• ORDER BY txn_date DESC, txn_id DESC: last transaction first.
• rn = 1: the last transaction of each month.

INTUITION: Last record per sub-group: ROW_NUMBER PARTITION BY (group + time_bucket) ORDER BY time DESC, filter rn = 1.

PATTERN TIP: Last record per group = ROW_NUMBER ORDER BY time DESC = 1. Always add tiebreaker (txn_id DESC) when timestamps can be equal.

Q57. New vs Returning Customer Classification

Advanced
JP Morgan, Barclays

PROBLEM STATEMENT

For each month, classify customers as New (first ever order this month) or Returning, and count each type.

Table: Orders

Column Name	Type	Description
order_id	INT	Order ID
customer_id	INT	Customer ID
order_date	DATE	Order date

SOLUTION 1: MIN OVER PARTITION BY + month comparison

```
SELECT order_month,
       SUM(CASE WHEN is_new=1 THEN 1 ELSE 0 END) AS new_customers,
       SUM(CASE WHEN is_new=0 THEN 1 ELSE 0 END) AS returning_customers
FROM (
  SELECT DISTINCT customer_id,
         DATE_FORMAT(order_date,'%Y-%m') AS order_month,
```

```
        CASE WHEN DATE_FORMAT(
               MIN(order_date) OVER (PARTITION BY customer_id), '%Y-%m')
                  = DATE_FORMAT(order_date,'%Y-%m')
             THEN 1 ELSE 0 END AS is_new
  FROM Orders
) t
GROUP BY order_month
ORDER BY order_month;
```

Explanation:

• MIN(order_date) OVER (PARTITION BY customer_id): customer's first ever order date on every row.
• is_new=1 if first order falls in current month.
• GROUP BY month: count new vs returning per month.

INTUITION: New = first order in this month. MIN() OVER (PARTITION BY customer) gives first-order date per customer without collapsing rows.

PATTERN TIP: MIN() OVER (PARTITION BY): attaches group minimum to each row. Compare to current row value to identify first-occurrence rows.

Q58. Product Affinity - Frequently Bought Together

Advanced
JP Morgan

PROBLEM STATEMENT

Find the top 10 product pairs most frequently bought together in the same order.

Table: OrderItems

Column Name	Type	Description
order_id	INT	Order ID
customer_id	INT	Customer ID
product_id	INT	Product ID

SOLUTION 1: Self-join on order_id with < to avoid duplicates

```
SELECT a.product_id AS product1,
       b.product_id AS product2,
       COUNT(DISTINCT a.order_id) AS co_occurrences
FROM OrderItems a
JOIN OrderItems b
  ON a.order_id = b.order_id
  AND a.product_id < b.product_id
GROUP BY a.product_id, b.product_id
ORDER BY co_occurrences DESC
LIMIT 10;
```

Explanation:

• Self-join on order_id: products in the same order.
• a.product_id < b.product_id: each pair appears once, no self-pairs.
• COUNT DISTINCT order_id: orders containing both products.

INTUITION: Market basket analysis: self-join on order_id for co-occurrence. The < condition produces each pair exactly once.

PATTERN TIP: a.id < b.id in self-joins: eliminates duplicate pairs (A,B) and (B,A) and self-pairs (A,A). Standard pattern for symmetric relationships.

Q59. Department Budget Utilisation

Advanced
Barclays

PROBLEM STATEMENT

Compare actual expenses to budget per department. Flag departments over 90% utilisation.

Table: Budget

Column Name	Type	Description
dept_id	INT	Department ID
budget_amount	DECIMAL	Budget amount
fiscal_year	INT	Fiscal year

Table: Expenses

Column Name	Type	Description
exp_id	INT	Expense ID
dept_id	INT	Department ID
amount	DECIMAL	Expense amount
exp_date	DATE	Expense date

SOLUTION 1: LEFT JOIN + COALESCE + ratio + CASE flag

```
SELECT b.dept_id, b.budget_amount,
       COALESCE(SUM(e.amount), 0) AS total_expenses,
       ROUND(100 * COALESCE(SUM(e.amount),0) / b.budget_amount, 1) AS
utilization_pct,
       CASE WHEN COALESCE(SUM(e.amount),0) / b.budget_amount > 0.9
            THEN 'Over 90%' ELSE 'OK' END AS status
FROM Budget b
LEFT JOIN Expenses e ON b.dept_id = e.dept_id
  AND YEAR(e.exp_date) = b.fiscal_year
GROUP BY b.dept_id, b.budget_amount
ORDER BY utilization_pct DESC;
```

Explanation:

- LEFT JOIN: include depts with no expenses.
- YEAR filter in JOIN condition: match expenses to correct fiscal year.
- COALESCE(SUM,0): convert NULL (no expenses) to 0.
- Ratio > 0.9: flag high utilisation.

INTUITION: Budget vs actual: LEFT JOIN for all depts, COALESCE for NULLs, compute ratio, CASE for flagging.

PATTERN TIP: COALESCE(expr, fallback): returns first non-NULL value. After LEFT JOIN, aggregate may be NULL for unmatched rows. COALESCE(SUM(col), 0) converts to 0.

Q60. Customer Lifetime Value Ranking

Advanced
Barclays, JP Morgan

PROBLEM STATEMENT

Compute CLV per customer (total revenue, order count, avg order value) and rank customers by total revenue.

Table: Orders

Column Name	Type	Description
order_id	INT	Order ID
customer_id	INT	Customer ID
order_date	DATE	Order date
amount	DECIMAL	Order amount

SOLUTION 1: GROUP BY + RANK on SUM

```
SELECT customer_id,
       COUNT(*) AS total_orders,
       SUM(amount) AS total_revenue,
       ROUND(AVG(amount),2) AS avg_order_value,
       RANK() OVER (ORDER BY SUM(amount) DESC) AS clv_rank
FROM Orders
GROUP BY customer_id
ORDER BY clv_rank;
```

Explanation:

• Multiple aggregates in one GROUP BY: COUNT, SUM, AVG.
• RANK() OVER (ORDER BY SUM(amount) DESC): window function applied over grouped result.
• CLV rank = total revenue rank.

INTUITION: CLV: aggregate per customer, rank by value. Window functions can be applied on top of GROUP BY results.

PATTERN TIP: Window functions on GROUP BY: the window sees one row per group. RANK() on SUM(amount) ranks groups by their aggregated total.

SECTION 7

Analytical Patterns and Conditional Logic

Q61. Immediate Delivery Percentage

Medium
LeetCode, UBS

PROBLEM STATEMENT

What percentage of first orders are immediate (preferred delivery date = order date)? Round to 2 decimal places.

Table: Delivery

Column Name	Type	Description
delivery_id	INT	Delivery ID
customer_id	INT	Customer ID
order_date	DATE	Order date
customer_pref_delivery_date	DATE	Preferred delivery date

SOLUTION 1: First orders subquery + AVG boolean

```
SELECT ROUND(100 * AVG(order_date = customer_pref_delivery_date), 2)
       AS immediate_percentage
FROM Delivery
WHERE (customer_id, order_date) IN (
  SELECT customer_id, MIN(order_date)
  FROM Delivery GROUP BY customer_id
);
```

Explanation:

- Subquery: first order date per customer.
- Row-value IN: filter only first orders.
- MySQL boolean: (col1 = col2) returns 1 or 0.
- AVG of 1s and 0s = proportion. * 100 = percentage.

INTUITION: MySQL boolean: expressions like (a = b) evaluate to 1 (true) or 0 (false). AVG gives proportion. x 100 = percentage.

PATTERN TIP: MySQL: AVG(col1 = col2) computes fraction of rows where col1 equals col2. Compact percentage without explicit CASE WHEN.

Q62. Find Team Size using Window COUNT

Easy
LeetCode

PROBLEM STATEMENT

Find the team size of each employee without losing individual employee rows.

Table: Employee

Column Name	Type	Description
employee_id	INT	Employee ID
team_id	INT	Team ID

SOLUTION 1: COUNT(*) OVER PARTITION BY

```
SELECT employee_id,
       COUNT(*) OVER (PARTITION BY team_id) AS team_size
FROM Employee;
```

Explanation:

• COUNT(*) OVER (PARTITION BY team_id): team count attached to every row.
• No GROUP BY: all employee rows preserved.

INTUITION: Add group aggregate to each row without collapsing = window function. COUNT OVER PARTITION BY group.

PATTERN TIP: Window function keeps all rows. GROUP BY collapses rows. For add group total to every row: always window function.

Q63. Product Price at a Given Date

Medium
LeetCode

PROBLEM STATEMENT

Find price of each product on 2019-08-16. Products with no update before that date have default price 10.

Table: Products

Column Name	Type	Description
product_id	INT	Product ID
new_price	INT	New price after change
change_date	DATE	Date of price change

SOLUTION 1: Correlated subquery + IFNULL

```
SELECT DISTINCT p.product_id,
       IFNULL((
         SELECT new_price FROM Products
         WHERE product_id = p.product_id
           AND change_date <= '2019-08-16'
         ORDER BY change_date DESC LIMIT 1
       ), 10) AS price
FROM Products p;
```

Explanation:

• Correlated subquery: most recent price on or before target date.
• ORDER BY DESC LIMIT 1: the latest applicable price.
• IFNULL(..., 10): default 10 if no price change before target date.

INTUITION: Point-in-time price lookup: MAX applicable date, return corresponding value. IFNULL for default when no record exists.

PATTERN TIP: Point-in-time: SELECT value WHERE entity=X AND date<=target ORDER BY date DESC LIMIT 1.

Q64. Biggest Single Number

Easy
LeetCode

PROBLEM STATEMENT

Find the largest number that appears only once. Return NULL if none.

Table: MyNumbers

Column Name	Type	Description
num	INT	A number value

SOLUTION 1: HAVING COUNT=1 + outer MAX

```
SELECT MAX(num) AS num
FROM (
  SELECT num FROM MyNumbers
  GROUP BY num
  HAVING COUNT(*) = 1
) t;
```

Explanation:

- Inner query: numbers appearing exactly once (HAVING COUNT = 1).
- Outer MAX: largest single-occurrence number.
- MAX on empty set returns NULL automatically.

INTUITION: Two steps: filter single-occurrence values (HAVING COUNT=1), then find MAX. MAX on empty set returns NULL.

PATTERN TIP: MAX() on an empty set returns NULL automatically. No IFNULL needed for the empty-result case.

Q65. Contest Registration Percentage

Easy
LeetCode

PROBLEM STATEMENT

Find the % of users registered in each contest, rounded to 2 decimals. Order by % DESC then contest_id ASC.

Table: Users

Column Name	Type	Description
user_id	INT	User ID
user_name	VARCHAR	Username

Table: Register

Column Name	Type	Description
contest_id	INT	Contest ID
user_id	INT	FK to Users

SOLUTION 1: COUNT / scalar total * 100

```
SELECT contest_id,
       ROUND(COUNT(user_id) * 100.0 / (SELECT COUNT(*) FROM Users), 2) AS
percentage
FROM Register
GROUP BY contest_id
ORDER BY percentage DESC, contest_id ASC;
```

Explanation:

- Scalar subquery: total users - same value for all groups.
- COUNT(user_id) / total * 100: participation rate per contest.
- Secondary sort: contest_id ASC breaks ties.

INTUITION: Percentage = part / whole * 100. Whole from scalar subquery. Part from GROUP BY COUNT.

PATTERN TIP: Scalar subquery in SELECT returns one value used as a constant denominator. Efficient since it executes only once.

Q66. Query Quality and Poor Query Percentage

Easy
LeetCode

PROBLEM STATEMENT

Find quality = AVG(rating/position) and poor_query_percentage (% where rating < 3) per query_name.

Table: Queries

Column Name	Type	Description
query_name	VARCHAR	Query name
result	VARCHAR	Query result
position	INT	Position 1-500
rating	INT	Rating 1-5

SOLUTION 1: AVG + SUM CASE conditional rate

```
SELECT query_name,
       ROUND(AVG(rating / position), 2) AS quality,
       ROUND(100 * SUM(CASE WHEN rating < 3 THEN 1 ELSE 0 END) / COUNT(*), 2)
       AS poor_query_percentage
FROM Queries
WHERE query_name IS NOT NULL
GROUP BY query_name;
```

Explanation:

- AVG(rating/position): average of per-row quality ratio.

• SUM(CASE WHEN rating<3) / COUNT * 100: poor query rate.
• WHERE IS NOT NULL: exclude null query names.

INTUITION: Multiple aggregation metrics in one GROUP BY. Each expression computes independently over the same groups.

PATTERN TIP: Multiple SUM(CASE), AVG() expressions in same GROUP BY: all compute in one table scan.

Q67. Median Salary

Hard
JP Morgan, UBS

PROBLEM STATEMENT

Find the median salary. If even number of employees, return average of two middle values.

Table: Employee

Column Name	Type	Description
emp_id	INT	Employee ID
salary	INT	Employee salary

SOLUTION 1: ROW_NUMBER + FLOOR/CEIL median formula

```
SELECT AVG(salary) AS median
FROM (
  SELECT salary,
         ROW_NUMBER() OVER (ORDER BY salary) AS rn,
         COUNT(*) OVER () AS total
  FROM Employee
) t
WHERE rn IN (FLOOR((total+1)/2), CEIL((total+1)/2));
```

Explanation:
• ROW_NUMBER ORDER BY salary: rank from lowest to highest.
• COUNT(*) OVER (): total rows on every row.
• FLOOR/CEIL of (N+1)/2: middle position(s) for both odd and even N.
• AVG: for odd N both positions are same row; for even N averages two rows.

INTUITION: Median positions: FLOOR((N+1)/2) and CEIL((N+1)/2). AVG handles both odd (same row) and even (two rows).

PATTERN TIP: MySQL lacks PERCENTILE_CONT. Use ROW_NUMBER + FLOOR/CEIL. N=5: both positions=3. N=6: positions 3 and 4.

Q68. Cohort Revenue Analysis

Advanced
JP Morgan, Barclays

PROBLEM STATEMENT

Show total lifetime revenue grouped by customers acquisition month (month of their first ever order).

Table: Orders

Column Name	Type	Description
order_id	INT	Order ID
customer_id	INT	Customer ID
order_date	DATE	Order date
amount	DECIMAL	Order revenue

SOLUTION 1: MIN date subquery + JOIN all orders

```
SELECT DATE_FORMAT(first.first_order,'%Y-%m') AS acquisition_month,
       COUNT(DISTINCT o.customer_id) AS customers_acquired,
       SUM(o.amount) AS lifetime_revenue
FROM Orders o
JOIN (
  SELECT customer_id, MIN(order_date) AS first_order
  FROM Orders GROUP BY customer_id
) first ON o.customer_id = first.customer_id
GROUP BY acquisition_month
ORDER BY acquisition_month;
```

Explanation:

- Subquery: first order date per customer.
- JOIN back to all orders: attribute all future revenue to acquisition cohort.
- SUM(amount): total lifetime value per cohort.

INTUITION: Cohort analysis: (1) compute first event per user, (2) join all events back, (3) group by cohort. All future revenue attributed to acquisition month.

PATTERN TIP: Cohort SQL: MIN per user -> JOIN all activity -> GROUP BY cohort month.

Q69. Multi-Currency Portfolio Summary

Advanced
UBS

PROBLEM STATEMENT

Summarise each client portfolio in USD. Holdings have local prices in different currencies. Use FX table to convert.

Table: Holdings

Column Name	Type	Description
client_id	INT	Client ID
security_id	VARCHAR	Security symbol
quantity	INT	Units held
local_price	DECIMAL	Price in local currency
currency	VARCHAR	Currency code

Table: FXRates

Column Name	Type	Description
currency	VARCHAR	Currency code
usd_rate	DECIMAL	USD per 1 unit of this currency

SOLUTION 1: JOIN FX + SUM converted value

```
SELECT h.client_id,
       SUM(h.quantity * h.local_price * fx.usd_rate) AS portfolio_usd
FROM Holdings h
JOIN FXRates fx ON h.currency = fx.currency
GROUP BY h.client_id
ORDER BY portfolio_usd DESC;
```

Explanation:

- JOIN on currency: attach USD rate to each holding.
- quantity * local_price * usd_rate: USD market value per holding.
- SUM per client: total portfolio in USD.

INTUITION: Currency conversion: JOIN on currency code, multiply quantity * price * rate, SUM per client.

PATTERN TIP: Multi-currency aggregation: JOIN FX table by currency, compute converted value per row, then SUM per entity.

Q70. Transaction Category Pivot Report

Advanced
Barclays

PROBLEM STATEMENT

Create monthly pivot: separate columns for Food, Travel, Shopping, Utilities spending per customer.

Table: Transactions

Column Name	Type	Description
txn_id	INT	Transaction ID
customer_id	INT	Customer ID
txn_date	DATE	Transaction date
category	VARCHAR	Food/Travel/Shopping/Utilities
amount	DECIMAL	Transaction amount

SOLUTION 1: Manual PIVOT with SUM CASE per category

```
SELECT customer_id,
       DATE_FORMAT(txn_date,'%Y-%m') AS month,
       SUM(CASE WHEN category='Food'      THEN amount ELSE 0 END) AS food,
       SUM(CASE WHEN category='Travel'    THEN amount ELSE 0 END) AS travel,
       SUM(CASE WHEN category='Shopping'  THEN amount ELSE 0 END) AS shopping,
       SUM(CASE WHEN category='Utilities' THEN amount ELSE 0 END) AS utilities
FROM Transactions
```

```
GROUP BY customer_id, DATE_FORMAT(txn_date,'%Y-%m')
ORDER BY customer_id, month;
```

Explanation:
• One SUM(CASE WHEN category='X') per output column.
• GROUP BY customer + month: one row per customer per month.
• MySQL has no native PIVOT - this manual technique is standard.

INTUITION: MySQL PIVOT = SUM(CASE WHEN category='X' THEN value ELSE 0 END) for each category. Called conditional aggregation.

PATTERN TIP: Manual PIVOT: one SUM(CASE WHEN col='value' THEN metric ELSE 0 END) per desired output column.

SECTION 8

Financial Domain Queries II

Q71. Asset Allocation Drift Detection

Advanced
UBS

PROBLEM STATEMENT

Find clients where any asset class actual allocation differs more than 5 percentage points from their target.

Table: TargetAllocation

Column Name	Type	Description
client_id	INT	Client ID
asset_class	VARCHAR	Asset class name
target_pct	DECIMAL	Target allocation % (0-100)

Table: ActualHoldings

Column Name	Type	Description
client_id	INT	Client ID
asset_class	VARCHAR	Asset class name
market_value	DECIMAL	Current market value

SOLUTION 1: Compute actual pct + ABS difference filter

```
SELECT a.client_id, a.asset_class, t.target_pct,
       ROUND(100 * a.market_value / tot.total_val, 2) AS actual_pct,
       ROUND(ABS(100 * a.market_value / tot.total_val - t.target_pct), 2) AS
drift
FROM ActualHoldings a
JOIN TargetAllocation t
  ON a.client_id = t.client_id AND a.asset_class = t.asset_class
```

```
JOIN (
  SELECT client_id, SUM(market_value) AS total_val
  FROM ActualHoldings GROUP BY client_id
) tot ON a.client_id = tot.client_id
WHERE ABS(100 * a.market_value / tot.total_val - t.target_pct) > 5
ORDER BY drift DESC;
```

Explanation:
• Totals subquery: total portfolio value per client.
• Actual % = market_value / total * 100.
• JOIN TargetAllocation: get target % per asset class.
• ABS(actual - target) > 5: drift exceeds 5 percentage points.

INTUITION: Drift = |actual% - target%|. Pattern: compute totals, join for target, compute ratio, ABS difference, filter.

PATTERN TIP: ABS(x): absolute value. For drift analysis: ABS(actual_pct - target_pct) > threshold.

Q72. Interest Rate Sensitivity (Mortgage Formula)

Advanced
Barclays, UBS

PROBLEM STATEMENT

For each loan, calculate monthly payment at current rate and at rate + 1%. Show the sensitivity (difference). Formula: P * (r/12) / (1 - (1+r/12)^(-n)).

Table: Loans

Column Name	Type	Description
loan_id	INT	Loan ID
principal	DECIMAL	Loan principal amount
annual_rate	DECIMAL	Annual interest rate as decimal (e.g. 0.05)
term_months	INT	Loan term in months

SOLUTION 1: POW() mortgage formula both scenarios

```
SELECT loan_id, principal, annual_rate,
       ROUND(principal*(annual_rate/12)
             /(1-POW(1+annual_rate/12,-term_months)),2) AS current_payment,
       ROUND(principal*((annual_rate+0.01)/12)
             /(1-POW(1+(annual_rate+0.01)/12,-term_months)),2) AS
payment_plus_1pct,
       ROUND(
         principal*((annual_rate+0.01)/12)/(1-POW(1+(annual_rate+0.01)/12,-
term_months))
         - principal*(annual_rate/12)/(1-POW(1+annual_rate/12,-term_months))
       ,2) AS rate_sensitivity
FROM Loans
ORDER BY rate_sensitivity DESC;
```

Explanation:
• POW(base, exponent): MySQL power function.
• Mortgage formula applied at current rate and rate+1%.

• Sensitivity = payment difference between two scenarios.

INTUITION: Complex financial formulas in SQL use math functions: POW, EXP, LOG, SQRT, ROUND. Sensitivity = difference between two computed scenarios.

PATTERN TIP: MySQL math: POW(x,n), SQRT(x), EXP(x), LOG(x), ABS(x), CEILING(x), FLOOR(x). Enables complex financial calculations in-database.

Q73. Large Cash Transaction SAR Report

Advanced
Barclays

PROBLEM STATEMENT

Find accounts where total cash transactions exceed $10,000 on a single day (SAR trigger). Aggregate multiple transactions on the same day.

Table: CashTransactions

Column Name	Type	Description
txn_id	INT	Transaction ID
account_id	INT	Account ID
txn_date	DATE	Transaction date
amount	DECIMAL	Cash amount

SOLUTION 1: GROUP BY account + date, HAVING SUM > 10000

```
SELECT account_id, txn_date,
       COUNT(*) AS txn_count,
       SUM(amount) AS daily_total
FROM CashTransactions
GROUP BY account_id, txn_date
HAVING SUM(amount) > 10000
ORDER BY daily_total DESC;
```

Explanation:

• GROUP BY account + date: aggregate per account per day.
• COUNT(*): number of transactions that day.
• HAVING SUM > 10000: SAR threshold - daily total exceeds limit.

INTUITION: SAR reporting: sum per account per day. GROUP BY both account AND date for daily aggregation. HAVING for threshold.

PATTERN TIP: AML compliance: GROUP BY entity + date, HAVING SUM > threshold catches aggregated amounts. Multiple small transactions can trigger same as one large.

Q74. Credit Score Consecutive Decline Detection

Advanced
Barclays

PROBLEM STATEMENT

Find customers whose credit score declined for 3 or more consecutive months. Return customer_id and the streak start date.

Table: CreditScores

Column Name	Type	Description
customer_id	INT	Customer ID
score_month	DATE	Month (YYYY-MM-01 format)
credit_score	INT	Credit score that month

SOLUTION 1: LAG + island technique on declining months

```
SELECT customer_id,
       MIN(score_month) AS streak_start,
       COUNT(*) AS streak_length
FROM (
  SELECT customer_id, score_month,
         DATE_SUB(score_month, INTERVAL
           ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY score_month)
MONTH
         ) AS island_grp
  FROM (
    SELECT customer_id, score_month, credit_score,
           LAG(credit_score) OVER (PARTITION BY customer_id ORDER BY
score_month) AS prev
    FROM CreditScores
  ) t
  WHERE credit_score < prev
) t2
GROUP BY customer_id, island_grp
HAVING COUNT(*) >= 3
ORDER BY customer_id;
```

Explanation:

- LAG: get previous month score per customer.
- WHERE credit_score < prev: keep only declining months.
- Island technique: month - ROW_NUMBER month interval = constant for consecutive months.
- HAVING >= 3: streaks of 3+ months.

INTUITION: Consecutive condition: flag rows, filter to flagged rows, apply island technique to find consecutive groups, filter by group size.

PATTERN TIP: Islands for months: month - ROW_NUMBER (INTERVAL MONTH) = constant for consecutive calendar months. Core pattern for financial consecutive-period analysis.

Q75. Top Spending Customer per Month

Medium
Barclays, UBS

PROBLEM STATEMENT

Find the customer with the highest total spending in each month.

Table: Orders

Column Name	Type	Description
order_id	INT	Order ID
customer_id	INT	Customer ID
order_date	DATE	Order date
amount	DECIMAL	Order amount

SOLUTION 1: ROW_NUMBER on monthly spend aggregation

```
SELECT month, customer_id, monthly_spend
FROM (
  SELECT DATE_FORMAT(order_date,'%Y-%m') AS month,
         customer_id,
         SUM(amount) AS monthly_spend,
         ROW_NUMBER() OVER (
           PARTITION BY DATE_FORMAT(order_date,'%Y-%m')
           ORDER BY SUM(amount) DESC
         ) AS rn
  FROM Orders
  GROUP BY DATE_FORMAT(order_date,'%Y-%m'), customer_id
) t
WHERE rn = 1
ORDER BY month;
```

Explanation:

- GROUP BY month + customer: monthly spend per customer.
- ROW_NUMBER PARTITION BY month ORDER BY SUM DESC: rank within each month.
- rn = 1: top spender per month.

INTUITION: Top 1 per group after aggregation: GROUP BY, then ROW_NUMBER PARTITION BY group, filter rn = 1.

PATTERN TIP: ROW_NUMBER for exactly 1 result per group even with ties. DENSE_RANK if you want all tied top entries.

Q76. Insurance Premium Collection Efficiency

Advanced
Barclays

PROBLEM STATEMENT

Per insurance agent: total policies, total billed, total collected, collection rate %. Flag agents below 80%.

Table: Policies

Column Name	Type	Description
policy_id	INT	Policy ID
agent_id	INT	Agent ID
premium_due	DECIMAL	Premium amount due

Table: Payments

Column Name	Type	Description
payment_id	INT	Payment ID
policy_id	INT	FK to Policies
amount_paid	DECIMAL	Amount collected

SOLUTION 1: LEFT JOIN + COALESCE + ratio + flag

```
SELECT p.agent_id,
       COUNT(DISTINCT p.policy_id) AS total_policies,
       SUM(p.premium_due) AS total_billed,
       COALESCE(SUM(pay.amount_paid), 0) AS total_collected,
       ROUND(100 * COALESCE(SUM(pay.amount_paid),0) / SUM(p.premium_due), 2) AS
collection_rate,
       CASE WHEN COALESCE(SUM(pay.amount_paid),0) / SUM(p.premium_due) < 0.8
            THEN 'FLAG' ELSE 'OK' END AS status
FROM Policies p
LEFT JOIN Payments pay ON p.policy_id = pay.policy_id
GROUP BY p.agent_id
ORDER BY collection_rate;
```

Explanation:

- LEFT JOIN: include policies with no payments (uncollected).
- COALESCE(SUM,0): NULL from LEFT JOIN becomes 0.
- Collection rate = collected / billed * 100.

INTUITION: Collection efficiency: LEFT JOIN for all policies, COALESCE for NULLs, ratio, flag.

PATTERN TIP: COALESCE(expr, fallback): returns first non-NULL. After LEFT JOIN, SUM may be NULL for unmatched rows.

Q77. Cross-Sell: Customers with Product A but not B

Medium
Barclays, UBS

PROBLEM STATEMENT

Find customers who bought product_id=1 but have never bought product_id=2 (cross-sell targets).

Table: OrderItems

Column Name	Type	Description
order_id	INT	Order ID
customer_id	INT	Customer ID
product_id	INT	Product ID

SOLUTION 1: WHERE product=1 + NOT EXISTS for product=2

```
SELECT DISTINCT customer_id
FROM OrderItems o1
WHERE product_id = 1
  AND NOT EXISTS (
    SELECT 1 FROM OrderItems o2
```

```
    WHERE o2.customer_id = o1.customer_id
      AND o2.product_id = 2
);
```

Explanation:
• Outer query: customers who bought product 1.
• NOT EXISTS: no record of buying product 2 for this customer.
• DISTINCT: each qualifying customer appears once.

INTUITION: Cross-sell: in set A but not in set B. WHERE for set A membership, NOT EXISTS for absence from set B.

PATTERN TIP: Set difference in SQL: 'in A but not B' = WHERE condition for A + NOT EXISTS subquery for B.

Q78. Find Overlapping Date Ranges

Medium
Barclays

PROBLEM STATEMENT

Find all pairs of events that overlap in time. Two ranges overlap if start1 <= end2 AND start2 <= end1.

Table: Events

Column Name	Type	Description
event_id	INT	Event ID
event_name	VARCHAR	Event name
start_date	DATE	Start date
end_date	DATE	End date

SOLUTION 1: Self-join with overlap condition

```
SELECT a.event_id AS event1, b.event_id AS event2,
       a.event_name AS name1, b.event_name AS name2
FROM Events a
JOIN Events b
  ON a.event_id < b.event_id
  AND a.start_date <= b.end_date
  AND b.start_date <= a.end_date;
```

Explanation:
• a.event_id < b.event_id: each pair returned once, no self-pairs.
• Overlap condition: A starts before B ends AND B starts before A ends.
• Both conditions together = the exact definition of date range overlap.

INTUITION: Two ranges [s1,e1] and [s2,e2] overlap if: s1<=e2 AND s2<=e1. Self-join with this condition finds all overlapping pairs.

PATTERN TIP: Date range overlap: s1<=e2 AND s2<=e1. Non-overlap (complement): e1<s2 OR e2<s1.

Q79. Salary Outlier Detection (Z-Score)

Advanced
UBS, JP Morgan

PROBLEM STATEMENT

Find employees whose salary deviates from the company average by more than 2 standard deviations.

Table: Employee

Column Name	Type	Description
emp_id	INT	Employee ID
name	VARCHAR	Employee name
salary	INT	Salary

SOLUTION 1: AVG + STDDEV window functions

```
SELECT emp_id, name, salary,
       ROUND(avg_salary, 2) AS company_avg,
       ROUND(std_salary, 2) AS company_std,
       ROUND((salary - avg_salary) / NULLIF(std_salary, 0), 2) AS z_score
FROM (
  SELECT emp_id, name, salary,
         AVG(salary) OVER () AS avg_salary,
         STDDEV(salary) OVER () AS std_salary
  FROM Employee
) t
WHERE ABS(salary - avg_salary) > 2 * std_salary
ORDER BY ABS(salary - avg_salary) DESC;
```

Explanation:

- AVG() OVER (): company-wide average on every row.
- STDDEV() OVER (): company-wide standard deviation.
- z_score = (salary - mean) / std: standard score.
- NULLIF(std,0): prevent division by zero if all salaries are identical.
- ABS > 2 * std: more than 2 standard deviations away = outlier.

INTUITION: Statistical outlier: z-score = (value - mean) / std. |z| > 2 = outlier. STDDEV() window function computes this cleanly.

PATTERN TIP: STDDEV() is available as window function in MySQL: STDDEV(col) OVER (). NULLIF(x, 0) prevents divide-by-zero: returns NULL if x=0, else x.

Q80. Identify Gaps in Sequential IDs

Hard
Barclays, UBS

PROBLEM STATEMENT

Find all missing ID ranges in a sequential table where IDs should be consecutive.

Table: Records

Column Name	Type	Description
id	INT	Sequential ID (may have gaps)
data	VARCHAR	Record data

SOLUTION 1: LEAD to detect gaps

```
SELECT id + 1 AS gap_start,
       next_id - 1 AS gap_end,
       next_id - id - 1 AS missing_count
FROM (
  SELECT id,
         LEAD(id) OVER (ORDER BY id) AS next_id
  FROM Records
) t
WHERE next_id - id > 1;
```

Explanation:

- LEAD(id): the next ID in sequence.
- next_id - id > 1: gap exists between consecutive rows.
- id+1 to next_id-1: the range of missing IDs.

INTUITION: Gap detection: LEAD gets next value. Difference > 1 means gap. Missing range = current+1 to next-1.

PATTERN TIP: LEAD(id) - id = 1: consecutive. > 1: gap exists. LEAD returns NULL for the last row (no gap check needed there).

SECTION 9

Risk, Compliance and Operations SQL

Q81. 30-Day Rolling Fraud Flag Count

Advanced
Barclays

PROBLEM STATEMENT

For each transaction, count how many fraud-flagged transactions exist in the 30-day window before it for the same account.

Table: Transactions

Column Name	Type	Description
txn_id	INT	Transaction ID
account_id	INT	Account ID
txn_date	DATE	Transaction date
is_fraud	INT	1 = fraud, 0 = normal

SOLUTION 1: Self-join 30-day window + SUM is_fraud

```
SELECT t1.txn_id, t1.account_id, t1.txn_date,
       COALESCE(SUM(t2.is_fraud), 0) AS fraud_flags_30d
FROM Transactions t1
LEFT JOIN Transactions t2
  ON t1.account_id = t2.account_id
  AND t2.txn_date < t1.txn_date
  AND DATEDIFF(t1.txn_date, t2.txn_date) <= 30
GROUP BY t1.txn_id, t1.account_id, t1.txn_date
ORDER BY t1.account_id, t1.txn_date;
```

Explanation:

- LEFT JOIN on same account: prior transactions in 30-day window.
- t2.txn_date < t1.txn_date: strictly before current transaction.
- DATEDIFF <= 30: within 30 days.
- SUM(t2.is_fraud): count prior fraud flags in window.

INTUITION: Rolling prior-period count: self-join on same entity + DATEDIFF range (strictly before). SUM the flagged column.

PATTERN TIP: Rolling prior window: t2.txn_date < t1.txn_date AND DATEDIFF <= N. LEFT JOIN so accounts with no prior transactions get 0.

Q82. RFM Customer Segmentation

Advanced
JP Morgan, Barclays

PROBLEM STATEMENT

Compute RFM: R = days since last order, F = order count, M = total spend. Score each dimension 1-5 using NTILE.

Table: Orders

Column Name	Type	Description
order_id	INT	Order ID
customer_id	INT	Customer ID
order_date	DATE	Order date
amount	DECIMAL	Order amount

SOLUTION 1: GROUP BY + three NTILE window scores

```
SELECT customer_id,
       DATEDIFF(CURDATE(), MAX(order_date)) AS recency_days,
       COUNT(*) AS frequency,
       SUM(amount) AS monetary,
       NTILE(5) OVER (ORDER BY DATEDIFF(CURDATE(),MAX(order_date)) ASC)  AS
r_score,
       NTILE(5) OVER (ORDER BY COUNT(*) DESC)                            AS
f_score,
       NTILE(5) OVER (ORDER BY SUM(amount) DESC)                         AS
m_score
FROM Orders
GROUP BY customer_id;
```

Explanation:
• R: days since last order. Lower days = better = ORDER BY ASC (score 5 = most recent).
• F: order count. Higher = better = ORDER BY DESC (score 5 = most frequent).
• M: total spend. Higher = better = ORDER BY DESC.
• NTILE(5): divide into 5 equal buckets.

INTUITION: RFM: three grouped metrics per customer, each scored 1-5 with NTILE. R sorted ASC (lower days better), F and M sorted DESC.

PATTERN TIP: NTILE(5) ORDER BY DESC: bucket 1 = top 20%. For recency, ORDER BY ASC means bucket 5 = most recent (best).

Q83. Multi-Step Approval Chain Duration

Advanced
Barclays

PROBLEM STATEMENT

For each transaction, find the total hours between submission and final approval across a multi-step approval chain.

Table: ApprovalLog

Column Name	Type	Description
txn_id	INT	Transaction ID
step	VARCHAR	submitted / level1 / level2 / approved
log_time	DATETIME	Step timestamp

SOLUTION 1: MIN/MAX CASE timestamps + TIMESTAMPDIFF

```
SELECT txn_id,
       MIN(CASE WHEN step='submitted' THEN log_time END) AS submitted_at,
       MAX(CASE WHEN step='approved'  THEN log_time END) AS approved_at,
       TIMESTAMPDIFF(HOUR,
         MIN(CASE WHEN step='submitted' THEN log_time END),
         MAX(CASE WHEN step='approved'  THEN log_time END)
       ) AS hours_to_approve
FROM ApprovalLog
GROUP BY txn_id
HAVING MAX(CASE WHEN step='approved' THEN log_time END) IS NOT NULL
ORDER BY hours_to_approve DESC;
```

Explanation:
• MIN/MAX CASE: extract specific step timestamps per transaction.
• TIMESTAMPDIFF(HOUR, start, end): hours between submission and approval.
• HAVING IS NOT NULL: only transactions that reached approved status.

INTUITION: Event sourcing: use MIN/MAX CASE to pivot specific event timestamps, then TIMESTAMPDIFF for duration. Common in workflow analytics.

PATTERN TIP: MIN(CASE WHEN step='X' THEN timestamp END): extracts the timestamp for step X per entity. Works in GROUP BY.

Q84. Vendor Payment Terms Compliance

Advanced
Barclays

PROBLEM STATEMENT

Find vendors with more than 20% of invoices paid outside agreed terms (NET30 = within 30 days, NET60 = within 60 days).

Table: Invoices

Column Name	Type	Description
invoice_id	INT	Invoice ID
vendor_id	INT	Vendor ID
invoice_date	DATE	Invoice date
payment_date	DATE	Payment date
payment_terms	VARCHAR	NET30 or NET60

SOLUTION 1: Per-row rule CASE + conditional rate

```
SELECT vendor_id,
       COUNT(*) AS total_invoices,
       SUM(CASE
         WHEN payment_terms='NET30' AND DATEDIFF(payment_date,invoice_date)>30
THEN 1
         WHEN payment_terms='NET60' AND DATEDIFF(payment_date,invoice_date)>60
THEN 1
         ELSE 0 END) AS late_count,
       ROUND(100.0 * SUM(CASE
         WHEN payment_terms='NET30' AND DATEDIFF(payment_date,invoice_date)>30
THEN 1
         WHEN payment_terms='NET60' AND DATEDIFF(payment_date,invoice_date)>60
THEN 1
         ELSE 0 END) / COUNT(*), 2) AS late_pct
FROM Invoices
GROUP BY vendor_id
HAVING SUM(CASE
  WHEN payment_terms='NET30' AND DATEDIFF(payment_date,invoice_date)>30 THEN 1
  WHEN payment_terms='NET60' AND DATEDIFF(payment_date,invoice_date)>60 THEN 1
  ELSE 0 END) * 100.0 / COUNT(*) > 20
ORDER BY late_pct DESC;
```

Explanation:

- CASE handles different rules per row: NET30 vs NET60.
- SUM(CASE) / COUNT * 100: late payment rate.
- HAVING > 20: vendors with >20% late invoices.

INTUITION: Multi-rule compliance: CASE handles different rules per row type. Combine with conditional rate for percentage.

PATTERN TIP: When different rows have different rules, embed the rule check in CASE WHEN. CASE WHEN terms='NET30' AND days>30 applies per-row rule logic.

Q85. Sales Trend - Consistently Increasing Products

Advanced
JP Morgan, UBS

PROBLEM STATEMENT

Find products whose monthly sales have been consistently increasing for the last 3 months.

Table: MonthlySales

Column Name	Type	Description
product_id	INT	Product ID
sale_month	DATE	Month (YYYY-MM-01)
units_sold	INT	Units sold that month

SOLUTION 1: LAG 2 periods + all increasing check

```
SELECT product_id,
       ROUND(100.0*(latest - two_ago) / NULLIF(two_ago,0), 2) AS growth_pct_2m
FROM (
  SELECT product_id,
         units_sold AS latest,
         LAG(units_sold,1) OVER (PARTITION BY product_id ORDER BY sale_month) AS
one_ago,
         LAG(units_sold,2) OVER (PARTITION BY product_id ORDER BY sale_month) AS
two_ago
  FROM MonthlySales
) t
WHERE latest > one_ago AND one_ago > two_ago
ORDER BY growth_pct_2m DESC;
```

Explanation:

- LAG(1): previous month. LAG(2): two months ago.
- WHERE latest > one_ago AND one_ago > two_ago: consistently increasing.
- NULLIF(two_ago, 0): prevent division by zero.

INTUITION: Multi-period trend: LAG by 1 and 2 periods, check all three levels are increasing. NULLIF prevents divide-by-zero.

PATTERN TIP: NULLIF(x, 0): returns NULL if x=0, else x. Use before division to prevent divide-by-zero. NULL/anything = NULL (not an error).

Q86. Proportional Bonus Pool Allocation

Advanced
Barclays, UBS

PROBLEM STATEMENT

Allocate a $1,000,000 bonus pool proportionally to employees based on their performance score within each department.

Table: Employee

Column Name	Type	Description

emp_id	INT	Employee ID
name	VARCHAR	Employee name
dept_id	INT	Department ID
performance_score	DECIMAL	Performance score 0-100

SOLUTION 1: Proportional share with window SUM

```
SELECT emp_id, name, dept_id, performance_score,
       SUM(performance_score) OVER (PARTITION BY dept_id) AS dept_total_score,
       ROUND(1000000.0 * performance_score
             / SUM(performance_score) OVER (PARTITION BY dept_id), 2) AS
bonus_allocation
FROM Employee
ORDER BY dept_id, bonus_allocation DESC;
```

Explanation:

- SUM OVER (PARTITION BY dept_id): dept total score attached to every row.
- Individual share = score / dept_total.
- bonus = $1M * share.

INTUITION: Proportional allocation: share = individual_value / group_total. Window SUM gives group total without collapsing rows.

PATTERN TIP: Proportional split: value / SUM(value) OVER (PARTITION BY group). This distributes 100% of a pool proportionally within each group.

Q87. Detect Missing Monthly Reports

Advanced
Barclays

PROBLEM STATEMENT

Find branches that have not submitted a monthly report for at least one month between Jan 2023 and Dec 2023.

Table: MonthlyReports

Column Name	Type	Description
branch_id	INT	Branch ID
report_month	DATE	Month (YYYY-MM-01)
submitted	INT	1 if submitted, 0 if not

SOLUTION 1: CROSS JOIN expected + LEFT JOIN actual + IS NULL

```
SELECT b.branch_id, m.report_month
FROM (SELECT DISTINCT branch_id FROM MonthlyReports) b
CROSS JOIN (
  SELECT DISTINCT report_month FROM MonthlyReports
  WHERE report_month BETWEEN '2023-01-01' AND '2023-12-01'
) m
LEFT JOIN MonthlyReports r
  ON b.branch_id = r.branch_id
  AND m.report_month = r.report_month
  AND r.submitted = 1
```

```
WHERE r.branch_id IS NULL
ORDER BY b.branch_id, m.report_month;
```

Explanation:
• CROSS JOIN: all expected (branch, month) combinations.
• LEFT JOIN actual submitted reports: NULLs for missing or unsubmitted.
• IS NULL: find the missing (branch, month) pairs.

INTUITION: Missing expected records: CROSS JOIN generates all expected combinations, LEFT JOIN actuals, IS NULL finds gaps.

PATTERN TIP: Missing expected records pattern: CROSS JOIN all expected pairs -> LEFT JOIN fact table -> WHERE fact IS NULL. Classic gap-finding pattern.

Q88. Top 3 Merchant Categories per Cardholder

Advanced
Barclays, JP Morgan

PROBLEM STATEMENT

For each cardholder, find their top 3 spending merchant categories by total spend in the last 6 months.

Table: CardTransactions

Column Name	Type	Description
txn_id	INT	Transaction ID
cardholder_id	INT	Cardholder ID
mcc_code	VARCHAR	Merchant category code
amount	DECIMAL	Transaction amount
txn_date	DATE	Transaction date

SOLUTION 1: Aggregate + ROW_NUMBER per cardholder

```
SELECT cardholder_id, mcc_code, category_spend
FROM (
  SELECT cardholder_id, mcc_code,
         SUM(amount) AS category_spend,
         ROW_NUMBER() OVER (PARTITION BY cardholder_id ORDER BY SUM(amount)
DESC) AS rn
  FROM CardTransactions
  WHERE txn_date >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
  GROUP BY cardholder_id, mcc_code
) t
WHERE rn <= 3
ORDER BY cardholder_id, category_spend DESC;
```

Explanation:
• WHERE: last 6 months filter.
• GROUP BY cardholder + mcc: total spend per category per cardholder.
• ROW_NUMBER PARTITION BY cardholder ORDER BY SUM DESC: rank categories within each cardholder.
• rn <= 3: top 3 categories.

INTUITION: Top-N per group after aggregation: GROUP BY (entity + group), then ROW_NUMBER in outer query, filter rn <= N.

PATTERN TIP: Top-N after aggregation: aggregate first (GROUP BY), then apply ROW_NUMBER on aggregated result in outer query.

Q89. GL Journal Entry Reconciliation

Advanced
Barclays

PROBLEM STATEMENT

Find journal entry batches that are not balanced (total debits != total credits). Use 0.01 tolerance for floating point.

Table: JournalEntries

Column Name	Type	Description
entry_id	INT	Entry ID
journal_id	INT	Journal batch ID
account_code	VARCHAR	Account code
entry_type	VARCHAR	DEBIT or CREDIT
amount	DECIMAL	Entry amount

SOLUTION 1: SUM CASE netting + HAVING ABS > tolerance

```
SELECT journal_id,
       SUM(CASE WHEN entry_type='DEBIT'  THEN amount ELSE 0 END) AS
total_debits,
       SUM(CASE WHEN entry_type='CREDIT' THEN amount ELSE 0 END) AS
total_credits,
       SUM(CASE WHEN entry_type='DEBIT' THEN amount ELSE -amount END) AS
net_balance
FROM JournalEntries
GROUP BY journal_id
HAVING ABS(SUM(CASE WHEN entry_type='DEBIT' THEN amount ELSE -amount END)) >
0.01
ORDER BY ABS(net_balance) DESC;
```

Explanation:

- SUM CASE: separate debit and credit totals.
- Net balance = debits - credits (DEBIT positive, CREDIT negative).
- HAVING ABS > 0.01: unbalanced journals (tolerance for floating point).

INTUITION: Double-entry accounting: debits must equal credits per journal. Net = debits - credits = 0 for balanced. HAVING ABS > tolerance finds unbalanced.

PATTERN TIP: Floating point tolerance: use ABS > 0.01 instead of != 0. Floating point arithmetic can produce tiny non-zero values even for 'equal' amounts.

Q90. Predict Next Order Date

Advanced
JP Morgan, Barclays

PROBLEM STATEMENT

For each customer, predict their next order date based on their average order interval.

Table: Orders

Column Name	Type	Description
order_id	INT	Order ID
customer_id	INT	Customer ID
order_date	DATE	Order date

SOLUTION 1: LAG + AVG gap + DATE_ADD prediction

```
SELECT customer_id,
       MAX(order_date) AS last_order,
       ROUND(AVG(DATEDIFF(order_date, prev_date)), 0) AS avg_days_between,
       DATE_ADD(MAX(order_date),
         INTERVAL ROUND(AVG(DATEDIFF(order_date, prev_date)), 0) DAY
       ) AS predicted_next_order
FROM (
  SELECT customer_id, order_date,
         LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS
prev_date
  FROM Orders
) t
WHERE prev_date IS NOT NULL
GROUP BY customer_id
ORDER BY predicted_next_order;
```

Explanation:

- LAG: previous order date per customer.
- DATEDIFF: gap between consecutive orders.
- AVG gap: average inter-order interval.
- DATE_ADD last_order by avg interval: predicted next order date.

INTUITION: Order prediction: compute average inter-order interval using LAG + DATEDIFF, then DATE_ADD the last order by that average.

PATTERN TIP: DATE_ADD(date, INTERVAL expr DAY): add computed interval. INTERVAL must be integer so use ROUND() to convert decimal AVG.

SECTION 10

Advanced Assessment Level Questions

Q91. FX Options Portfolio Summary

Advanced
UBS

PROBLEM STATEMENT

Summarise FX options by underlying currency: total notional, net delta, count of calls vs puts.

Table: FXOptions

Column Name	Type	Description
option_id	INT	Option ID
underlying	VARCHAR	Currency pair (e.g. EURUSD)
option_type	VARCHAR	CALL or PUT
notional	DECIMAL	Notional amount
delta	DECIMAL	Delta sensitivity -1 to 1

SOLUTION 1: GROUP BY + SUM + CASE pivot for call/put

```
SELECT underlying,
       SUM(notional) AS total_notional,
       SUM(delta * notional) AS net_dollar_delta,
       SUM(CASE WHEN option_type='CALL' THEN 1 ELSE 0 END) AS call_count,
       SUM(CASE WHEN option_type='PUT'  THEN 1 ELSE 0 END) AS put_count
FROM FXOptions
GROUP BY underlying
ORDER BY total_notional DESC;
```

Explanation:

- SUM(delta * notional): dollar delta = delta-weighted notional.
- SUM(CASE) for CALL and PUT: conditional count pivot.

INTUITION: Options portfolio: dollar delta = delta * notional = risk contribution. CASE WHEN pivot for call/put breakdown.

PATTERN TIP: Dollar delta = delta * notional = risk contribution per option. SUM gives net risk. CASE pivot breaks down by type.

Q92. Employee Skill Gap Analysis

Advanced
Barclays, JP Morgan

PROBLEM STATEMENT

Find employees who are missing at least one required skill for their current role.

Table: RoleRequirements

Column Name	Type	Description
role_id	INT	Role ID
skill_id	INT	Required skill ID

Table: EmployeeSkills

Column Name	Type	Description

emp_id	INT	Employee ID
skill_id	INT	Skill ID the employee has

Table: Employees

Column Name	Type	Description
emp_id	INT	Employee ID
name	VARCHAR	Employee name
role_id	INT	Current role ID

SOLUTION 1: JOIN requirements + LEFT JOIN skills + gap COUNT

```
SELECT e.emp_id, e.name, e.role_id,
       COUNT(DISTINCT rr.skill_id) AS required_skills,
       COUNT(DISTINCT es.skill_id) AS skills_employee_has,
       COUNT(DISTINCT rr.skill_id) - COUNT(DISTINCT es.skill_id) AS skill_gap
FROM Employees e
JOIN RoleRequirements rr ON e.role_id = rr.role_id
LEFT JOIN EmployeeSkills es
  ON e.emp_id = es.emp_id AND rr.skill_id = es.skill_id
GROUP BY e.emp_id, e.name, e.role_id
HAVING COUNT(DISTINCT rr.skill_id) > COUNT(DISTINCT es.skill_id)
ORDER BY skill_gap DESC;
```

Explanation:

- JOIN RoleRequirements: get all required skills per employee's role.
- LEFT JOIN EmployeeSkills on BOTH emp_id AND skill_id: match possessed skills.
- Difference in DISTINCT counts = skill gap.
- HAVING gap > 0: employees missing at least one skill.

INTUITION: Skill gap = required count - possessed count. JOIN requirements, LEFT JOIN actual skills on both entity and skill ID.

PATTERN TIP: LEFT JOIN on multiple conditions (emp_id AND skill_id) matches the specific required skill to the employee's actual skills. Key for per-skill gap analysis.

Q93. OHLC Prices from Tick Data

Advanced
UBS, Barclays

PROBLEM STATEMENT

Compute Open, High, Low, Close prices for each security each day from tick-by-tick trade data.

Table: Ticks

Column Name	Type	Description
tick_id	INT	Tick ID
security_id	VARCHAR	Security symbol
tick_time	DATETIME	Trade timestamp
price	DECIMAL	Trade price at this tick

SOLUTION 1: MIN/MAX + FIRST_VALUE/LAST_VALUE with full frame

```
SELECT security_id,
       DATE(tick_time) AS trade_date,
       MIN(price) AS low_price,
       MAX(price) AS high_price,
       FIRST_VALUE(price) OVER (
         PARTITION BY security_id, DATE(tick_time)
         ORDER BY tick_time
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
open_price,
       LAST_VALUE(price) OVER (
         PARTITION BY security_id, DATE(tick_time)
         ORDER BY tick_time
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS
close_price
FROM Ticks
GROUP BY security_id, DATE(tick_time);
```

Explanation:

• MIN/MAX per security per day: Low and High.
• FIRST_VALUE: first tick of the day = Open.
• LAST_VALUE with UNBOUNDED frame: last tick = Close.
• LAST_VALUE REQUIRES the full ROWS BETWEEN frame - without it, it returns the current row value.

INTUITION: OHLC: Open=first price, High=MAX, Low=MIN, Close=last price. FIRST_VALUE and LAST_VALUE (with full frame) for open/close.

PATTERN TIP: LAST_VALUE trap: default frame ends at CURRENT ROW so LAST_VALUE returns current row value. MUST specify ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING for true last value.

Q94. Credit Utilisation Spike Detection

Advanced
Barclays

PROBLEM STATEMENT

Find customers whose most recent month credit utilisation is more than 20 percentage points above their prior months average.

Table: MonthlyUtilization

Column Name	Type	Description
customer_id	INT	Customer ID
util_month	DATE	Month (YYYY-MM-01)
utilization_pct	DECIMAL	Credit utilisation percentage 0-100

SOLUTION 1: Latest vs historical average comparison

```
SELECT customer_id,
       latest_util,
       ROUND(avg_prior_util, 2) AS avg_prior_util,
       ROUND(latest_util - avg_prior_util, 2) AS spike
```

```
FROM (
  SELECT customer_id,
         MAX(CASE WHEN util_month = MAX(util_month) OVER (PARTITION BY
customer_id)
                  THEN utilization_pct END) AS latest_util,
         AVG(CASE WHEN util_month < MAX(util_month) OVER (PARTITION BY
customer_id)
                  THEN utilization_pct END) AS avg_prior_util
  FROM MonthlyUtilization
  GROUP BY customer_id
) t
WHERE latest_util - avg_prior_util > 20
ORDER BY spike DESC;
```

Explanation:

- MAX(util_month) OVER (PARTITION BY customer_id): latest month per row.
- MAX(CASE ... = latest): extract current utilisation.
- AVG(CASE ... < latest): average of all prior months.
- Difference > 20: utilisation spike threshold.

INTUITION: Spike detection: compare latest value to historical average. Window MAX identifies latest month, CASE expressions isolate current vs historical.

PATTERN TIP: MAX(CASE WHEN col = MAX(col) OVER (PARTITION BY g) THEN value END): extracts the value at the latest record per group. Advanced but very powerful pattern.

Q95. GL Entry Balance Check

Advanced
Barclays

PROBLEM STATEMENT

Find journal entry batches where debits do not equal credits (accounting reconciliation). Show the imbalance amount.

Table: JournalEntries

Column Name	Type	Description
entry_id	INT	Entry ID
journal_id	INT	Journal batch ID
entry_type	VARCHAR	DEBIT or CREDIT
amount	DECIMAL	Entry amount
account_code	VARCHAR	Account code

SOLUTION 1: SUM CASE netting per journal

```
SELECT journal_id,
       SUM(CASE WHEN entry_type='DEBIT'  THEN amount ELSE 0 END) AS
total_debits,
       SUM(CASE WHEN entry_type='CREDIT' THEN amount ELSE 0 END) AS
total_credits,
       ABS(SUM(CASE WHEN entry_type='DEBIT' THEN amount ELSE -amount END)) AS
imbalance
FROM JournalEntries
GROUP BY journal_id
```

```
HAVING ABS(SUM(CASE WHEN entry_type='DEBIT' THEN amount ELSE -amount END)) >
0.005
ORDER BY imbalance DESC;
```

Explanation:

- SUM CASE: separate totals for DEBIT and CREDIT.
- Net = DEBIT - CREDIT. ABS gives magnitude.
- HAVING > 0.005: strict decimal tolerance for accounting precision.

INTUITION: Accounting balance check: net = debits - credits. Zero = balanced. Non-zero = reconciliation item. ABS for magnitude.

PATTERN TIP: Accounting reconciliation in SQL: SUM(CASE WHEN DEBIT THEN amount ELSE -amount END) per journal. HAVING ABS > small_tolerance finds imbalances.

Q96. Customer Wallet Share Analysis

Advanced
JP Morgan

PROBLEM STATEMENT

For each customer, compute wallet share = our deposits / estimated total wealth. Flag customers with share below 30%.

Table: CustomerDeposits

Column Name	Type	Description
customer_id	INT	Customer ID
account_type	VARCHAR	Account type
balance	DECIMAL	Current balance

Table: CustomerProfile

Column Name	Type	Description
customer_id	INT	Customer ID
estimated_total_wealth	DECIMAL	Estimated total wealth across all banks

SOLUTION 1: SUM deposits + ratio vs estimated wealth

```
SELECT d.customer_id,
       SUM(d.balance) AS our_deposits,
       p.estimated_total_wealth,
       ROUND(100 * SUM(d.balance) / p.estimated_total_wealth, 2) AS
wallet_share_pct,
       CASE WHEN SUM(d.balance) / p.estimated_total_wealth < 0.3
            THEN 'Opportunity' ELSE 'Loyal' END AS segment
FROM CustomerDeposits d
JOIN CustomerProfile p ON d.customer_id = p.customer_id
GROUP BY d.customer_id, p.estimated_total_wealth
ORDER BY wallet_share_pct;
```

Explanation:

- SUM(d.balance): total deposits with us per customer.

• / estimated_total_wealth: wallet share.
• < 0.3 (30%): low wallet share = cross-sell opportunity.

INTUITION: Wallet share = our deposits / total wealth. JOIN deposit aggregate to profile. Flag low share as opportunity.

PATTERN TIP: Wallet share analysis: GROUP BY customer for deposit total, JOIN profile for wealth, compute ratio, CASE for segment.

Q97. Daily Trader P&L (Realised and Unrealised)

Advanced
UBS, Barclays

PROBLEM STATEMENT

Calculate daily realised P&L (closed positions) and unrealised P&L (open positions marked to market) per trader.

Table: Positions

Column Name	Type	Description
position_id	INT	Position ID
trader_id	INT	Trader ID
trade_date	DATE	Trade date
quantity	INT	Units held
entry_price	DECIMAL	Entry price
close_price	DECIMAL	Close price (NULL if still open)
market_price	DECIMAL	Current market price

SOLUTION 1: SUM CASE for realised vs unrealised split

```
SELECT trader_id, trade_date,
       SUM(CASE WHEN close_price IS NOT NULL
                THEN quantity * (close_price - entry_price) ELSE 0 END) AS
realised_pnl,
       SUM(CASE WHEN close_price IS NULL
                THEN quantity * (market_price - entry_price) ELSE 0 END) AS
unrealised_pnl
FROM Positions
GROUP BY trader_id, trade_date
ORDER BY trader_id, trade_date;
```

Explanation:
• close_price IS NOT NULL: closed position -> realised P&L.
• close_price IS NULL: open position -> unrealised P&L = mark-to-market.
• P&L formula: quantity * (exit - entry).

INTUITION: P&L = quantity * (exit - entry). CASE on IS NULL / IS NOT NULL splits by position status. SUM aggregates per trader per day.

PATTERN TIP: Realised: quantity * (close - entry). Unrealised: quantity * (market - entry). IS NULL on close_price distinguishes open from closed.

Q98. Mortgage Arrears Detection

Advanced
Barclays

PROBLEM STATEMENT

Identify mortgages in arrears: 2 or more consecutive monthly payment misses. Return loan_id and streak start date.

Table: MortgagePayments

Column Name	Type	Description
loan_id	INT	Loan ID
customer_id	INT	Customer ID
payment_month	DATE	Payment month (YYYY-MM-01)
paid	INT	1 = paid, 0 = missed

SOLUTION 1: Filter missed + island technique

```
SELECT loan_id, customer_id,
       MIN(payment_month) AS arrears_start,
       COUNT(*) AS consecutive_missed
FROM (
  SELECT loan_id, customer_id, payment_month,
         DATE_SUB(payment_month,
           INTERVAL ROW_NUMBER() OVER (PARTITION BY loan_id ORDER BY
payment_month) MONTH
         ) AS island_grp
  FROM MortgagePayments
  WHERE paid = 0
) t
GROUP BY loan_id, customer_id, island_grp
HAVING COUNT(*) >= 2
ORDER BY consecutive_missed DESC;
```

Explanation:

- WHERE paid=0: only missed payments.
- Island technique: payment_month - ROW_NUMBER months = constant for consecutive missed months.
- GROUP BY island: each consecutive run = one group.
- HAVING >= 2: 2+ consecutive misses.

INTUITION: Mortgage arrears: filter missed payments, island technique to find consecutive runs, filter by run length.

PATTERN TIP: Islands for months: month - ROW_NUMBER (INTERVAL MONTH) = constant for consecutive calendar months. Core financial consecutive-period pattern.

Q99. ATM Network Performance Dashboard

Advanced
Barclays

PROBLEM STATEMENT

For each ATM: total transactions, avg transaction amount, uptime % (success/total), and peak hour (hour with most transactions).

Table: ATMTransactions

Column Name	Type	Description
txn_id	INT	Transaction ID
atm_id	INT	ATM ID
txn_datetime	DATETIME	Transaction timestamp
amount	DECIMAL	Withdrawal amount
status	VARCHAR	success or error

SOLUTION 1: Multiple aggregates + correlated subquery for mode

```
SELECT atm_id,
       COUNT(*) AS total_txns,
       ROUND(AVG(amount), 2) AS avg_amount,
       ROUND(100 * SUM(CASE WHEN status='success' THEN 1 ELSE 0 END) / COUNT(*),
2) AS uptime_pct,
       (SELECT HOUR(txn_datetime)
        FROM ATMTransactions t2
        WHERE t2.atm_id = t1.atm_id
        GROUP BY HOUR(txn_datetime)
        ORDER BY COUNT(*) DESC
        LIMIT 1) AS peak_hour
FROM ATMTransactions t1
GROUP BY atm_id
ORDER BY total_txns DESC;
```

Explanation:

- Multiple aggregates: COUNT, AVG, conditional SUM for uptime.
- Correlated subquery: per ATM find the hour with most transactions.
- HOUR(txn_datetime): extracts hour 0-23 from datetime.

INTUITION: Dashboard: combine multiple aggregates in one GROUP BY. Use correlated subquery for the mode (most frequent value per group).

PATTERN TIP: Finding mode per group: correlated subquery GROUP BY value, ORDER BY COUNT(*) DESC, LIMIT 1. Attach to outer via entity ID.

Q100. Composite Fraud Scorecard

Advanced
Barclays, JP Morgan

PROBLEM STATEMENT

Build fraud scorecard per account: velocity flag (>10 txns in last 7 days), night transactions flag (>30% between 10pm-6am), round number flag (>50% are multiples of 1000). Flag accounts with 2+ risk flags.

Table: Transactions

Column Name	Type	Description
txn_id	INT	Transaction ID
account_id	INT	Account ID
txn_datetime	DATETIME	Transaction timestamp
amount	DECIMAL	Transaction amount

SOLUTION 1: Multi-feature CASE flags + composite score

```
SELECT account_id,
       COUNT(*) AS total_txns,
       SUM(CASE WHEN DATEDIFF(CURDATE(),DATE(txn_datetime))<=7 THEN 1 ELSE 0
END) AS txns_last_7d,
       ROUND(100.0 * SUM(CASE WHEN HOUR(txn_datetime)>=22 OR
HOUR(txn_datetime)<6
                              THEN 1 ELSE 0 END) / COUNT(*), 2) AS night_pct,
       ROUND(100.0 * SUM(CASE WHEN amount % 1000 = 0
                              THEN 1 ELSE 0 END) / COUNT(*), 2) AS
round_num_pct,
       -- Composite score: sum of triggered flags
       (CASE WHEN SUM(CASE WHEN DATEDIFF(CURDATE(),DATE(txn_datetime))<=7
                           THEN 1 ELSE 0 END) > 10 THEN 1 ELSE 0 END
       +CASE WHEN 100.0 * SUM(CASE WHEN HOUR(txn_datetime)>=22 OR
HOUR(txn_datetime)<6
                                   THEN 1 ELSE 0 END) / COUNT(*) > 30 THEN 1
ELSE 0 END
       +CASE WHEN 100.0 * SUM(CASE WHEN amount%1000=0
                                   THEN 1 ELSE 0 END) / COUNT(*) > 50 THEN 1
ELSE 0 END
       ) AS fraud_score
FROM Transactions
GROUP BY account_id
HAVING (CASE WHEN SUM(CASE WHEN DATEDIFF(CURDATE(),DATE(txn_datetime))<=7
                           THEN 1 ELSE 0 END) > 10 THEN 1 ELSE 0 END
       +CASE WHEN 100.0 * SUM(CASE WHEN HOUR(txn_datetime)>=22 OR
HOUR(txn_datetime)<6
                                   THEN 1 ELSE 0 END) / COUNT(*) > 30 THEN 1
ELSE 0 END
       +CASE WHEN 100.0 * SUM(CASE WHEN amount%1000=0
                                   THEN 1 ELSE 0 END) / COUNT(*) > 50 THEN 1
ELSE 0 END
       ) >= 2
ORDER BY fraud_score DESC;
```

Explanation:

- Velocity: DATEDIFF <= 7 days.
- Night: HOUR >= 22 OR HOUR < 6 (10pm to 6am window).
- Round number: amount % 1000 = 0.
- Each CASE adds 0 or 1 to composite score.
- HAVING score >= 2: accounts triggering 2+ risk flags.

INTUITION: Fraud scorecard = sum of binary feature flags. Each feature: SUM(CASE) for conditional count. Composite = total flags. HAVING filters high-risk accounts.

PATTERN TIP: Composite scorecard: sum of binary CASE flags. Simple but powerful risk scoring. Repeating expression in HAVING needed as SELECT aliases not usable there.

END OF SQL INTERVIEW PREPARATION GUIDE

150 Questions | MySQL | Barclays • JP Morgan • UBS

Good luck with your interviews and assessments!